



BRAVA LABS

Core Auth Changes Security Assessment Report

Version: 3.0

July, 2026

Contents

Introduction	3
Disclaimer	3
Document Structure	3
Overview	3
Security Assessment Summary	4
Scope	4
Approach	4
Coverage Limitations	5
Findings Summary	5
Detailed Findings	6
Summary of Findings	7
Nested Bundle Execution Causes Double Gas Refund Accounting	9
Same-Safe Reentrancy Can Execute Later Bundles Mid-Sequence	12
Atomic Cross Chain CCTP Bundle Execution Can Be Griefed	15
Bundle Execution Reverts Prevent Executors From Receiving Gas Refunds	18
Gas Refund Mechanism Fails To Account For Full CCTP Relay Execution Costs	20
Emergency Withdrawals Can Be Used To Bypass Brava Fees	21
CEI Violation In Fee Processing Enables Double-Fee Reentrancy	23
proposeToken Overwrite Resets Delay Timer	24
Emergency ETH Withdrawals May Revert For Smart Contract Owners	25
Cosigner Owners Can Make Signing Threshold Unreachable	27
Raw Revert Bubbling Can Spoof Module Errors	29
Bundle Signatures Do Not Bind Auth Config Version	31
Unbounded Auth Version Can Block Future Updates	33
Stale proposedDelay Can Override A Later Delay Increase	35
Accumulating ERC20 Allowance In BuyCover	37
Missing USDC Token Validation In CCTPBridgeSend	38
Swap Contracts Omit tokenIn Registry Validation	39
Proposal Overwrite Resets Timelock In proposeAction And proposePool	40
AcrossV3Supply Logs Zero sharesAfter On Fee-Only Call	41
Accumulating Token Allowance In ParaswapSwap	42
Inverted proposedDelay Guard In changeDelay	43
logAdminVaultEvent Lacks Caller Verification	44
Missing try/catch On decimals() In Gas Adaptor	45
bytes4 Pool ID Truncation Enables Collision	46
Missing Domain Name And Version Validation In initializeConfig	47
GasRefundAction Fee-On-Transfer Discrepancy	48
ParaswapSwap Source Token Not Validated Against params.tokenIn	49
Co-Signers Can Be Configured With Zero Threshold	50
OP Stack L1 Fee Is Calculated From Incorrect Input Bytes	52
AcrossV3 Fee Timestamp Key Mismatch	54
setActionConfig Allows Immediate Redirection Of Protocol Endpoints	55
Fee Range Changes Can Affect Already Signed Transactions	56
ArbitrumGasPriceAdaptor Omits Fixed L1 Fee Component	58
getActionConfig Error Omits protocolName	59
cancelFeeConfigProposal Missing Existence Check	60
GasRefundAction Constructor Missing Zero-Address Check	61
PullToken Lacks Delegatecall Context Guard	62
ParaswapSwap Calldata Length Validation Insufficient	63
AssignToken Unvalidated balanceBefore	64
Miscellaneous General Comments	65

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the Brava Labs components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the Brava Labs components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the Brava Labs components in scope.

Overview

The Brava protocol is a set of pools built on top of the Safe smart wallet platform that enable the use of self-custodied investment strategies, intended for consumption by institutional clients as well as less technical end-users.

Brava achieves this with a system of pools with delegated actions that act directly on each Safe smart wallet, avoiding the need to combine user funds or create undesirable trust assumptions.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [Brava Labs](#) repository. This review contained two cycles of codebase review:

The scope of the first round of the time-boxed review was strictly limited to the following files, assessed at commit [01d8274](#):

- | | |
|---|---|
| 1. <i>AuthRegistry.sol</i> | 7. <i>Errors.sol</i> |
| 2. <i>BravaModuleLookup.sol</i> | 8. <i>IAuthRegistry.sol</i> |
| 3. <i>CCTPBridgeSend.sol</i> | 9. <i>IBravaSafeModule.sol</i> |
| 4. <i>EIP712TypedDataLib.sol</i> | 10. <i>IEip712TypedDataSafeModule.sol</i> |
| 5. <i>EIP712TypedDataSafeModule.sol</i> | 11. <i>IEmergencyWithdrawModule.sol</i> |
| 6. <i>EmergencyWithdrawModule.sol</i> | 12. <i>IMorphoBlue.sol</i> |

For the second round, the original scope was re-assessed at commit [017f1345](#). Additionally, an LLM assisted scan was run on the entire codebase at the same commit.

Finally, remaining resolutions were assessed at commit [826d0ab](#).

Note: third party libraries and dependencies were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 40 issues during this assessment. Categorised by their severity:

- High: 2 issues.
- Medium: 3 issues.
- Low: 22 issues.
- Informational: 13 issues.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the Brava Labs components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open:** the finding has not been addressed by the project team.
- **Resolved:** the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed:** the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
BRV4-01	Nested Bundle Execution Causes Double Gas Refund Accounting	High	Resolved
BRV4-02	Same-Safe Reentrancy Can Execute Later Bundles Mid-Sequence	High	Resolved
BRV4-03	Atomic Cross Chain CCTP Bundle Execution Can Be Griefed	Medium	Resolved
BRV4-04	Bundle Execution Reverts Prevent Executors From Receiving Gas Refunds	Medium	Closed
BRV4-05	Gas Refund Mechanism Fails To Account For Full CCTP Relay Execution Costs	Medium	Resolved
BRV4-06	Emergency Withdrawals Can Be Used To Bypass Brava Fees	Low	Closed
BRV4-07	CEI Violation In Fee Processing Enables Double-Fee Reentrancy	Low	Resolved
BRV4-08	proposeToken Overwrite Resets Delay Timer	Low	Closed
BRV4-09	Emergency ETH Withdrawals May Revert For Smart Contract Owners	Low	Closed
BRV4-10	Cosigner Owners Can Make Signing Threshold Unreachable	Low	Closed
BRV4-11	Raw Revert Bubbling Can Spoof Module Errors	Low	Resolved
BRV4-12	Bundle Signatures Do Not Bind Auth Config Version	Low	Closed
BRV4-13	Unbounded Auth Version Can Block Future Updates	Low	Resolved
BRV4-14	Stale proposedDelay Can Override A Later Delay Increase	Low	Closed
BRV4-15	Accumulating ERC20 Allowance In BuyCover	Low	Resolved
BRV4-16	Missing USDC Token Validation In CCTPBridgeSend	Low	Resolved
BRV4-17	Swap Contracts Omit tokenIn Registry Validation	Low	Resolved
BRV4-18	Proposal Overwrite Resets Timelock In proposeAction And proposePool	Low	Closed
BRV4-19	AcrossV3Supply Logs Zero sharesAfter On Fee-Only Call	Low	Resolved
BRV4-20	Accumulating Token Allowance In ParaswapSwap	Low	Resolved
BRV4-21	Inverted proposedDelay Guard In changeDelay	Low	Closed
BRV4-22	logAdminVaultEvent Lacks Caller Verification	Low	Resolved
BRV4-23	Missing try/catch On decimals() In Gas Adaptor	Low	Resolved
BRV4-24	bytes4 Pool ID Truncation Enables Collision	Low	Closed
BRV4-25	Missing Domain Name And Version Validation In initializeConfig	Low	Resolved

BRV4-26	GasRefundAction Fee-On-Transfer Discrepancy	Low	Resolved
BRV4-27	ParaswapSwap Source Token Not Validated Against <code>params.tokenIn</code>	Low	Resolved
BRV4-28	Co-Signers Can Be Configured With Zero Threshold	Informational	Closed
BRV4-29	OP Stack L1 Fee Is Calculated From Incorrect Input Bytes	Informational	Closed
BRV4-30	AcrossV3 Fee Timestamp Key Mismatch	Informational	Resolved
BRV4-31	setActionConfig Allows Immediate Redirection Of Protocol Endpoints	Informational	Closed
BRV4-32	Fee Range Changes Can Affect Already Signed Transactions	Informational	Closed
BRV4-33	ArbitrumGasPriceAdaptor Omits Fixed L1 Fee Component	Informational	Closed
BRV4-34	getActionConfig Error Omits <code>protocolName</code>	Informational	Closed
BRV4-35	cancelFeeConfigProposal Missing Existence Check	Informational	Closed
BRV4-36	GasRefundAction Constructor Missing Zero-Address Check	Informational	Resolved
BRV4-37	PullToken Lacks Delegatecall Context Guard	Informational	Resolved
BRV4-38	ParaswapSwap Calldata Length Validation Insufficient	Informational	Resolved
BRV4-39	AssignToken Unvalidated <code>balanceBefore</code>	Informational	Resolved
BRV4-40	Miscellaneous General Comments	Informational	Closed

BRV4-01	Nested Bundle Execution Causes Double Gas Refund Accounting		
Assets	auth/EIP712TypedDataSafeModule.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

Executors can receive refunds exceeding the actual gas consumed when bundle execution triggers a nested invocation of `executeBundle()`, causing the same gas usage to be accounted for multiple times.

The gas refund mechanism records the available gas at the beginning of each `executeBundle()` execution and later refunds the difference between the recorded value and the remaining gas at the point the refund action is executed.

```
function executeBundle(
    address _safeAddr,
    ITyped.Bundle calldata _bundle,
    bytes calldata _signatures
) external onlyInitialized {
    uint256 gasStart = gasleft();

    //....

    _executeChainSequence(_safeAddr, _bundle, _signatures, expectedSequenceNonce, gasStart, targetSeq);
}

function _executeChainSequence(
    address _safeAddr,
    ITyped.Bundle calldata _bundle,
    bytes calldata _signatures,
    uint256 _expectedSequenceNonce,
    uint256 _gasStart,
    ITyped.ChainSequence memory targetSequence
) internal {
    //....

    _execThroughSafeOrRevert(
        _safeAddr,
        ISequenceExecutor.Sequence({
            name: targetSequence.sequence.name,
            callData: targetSequence.sequence.callData,
            actionIds: actionIds
        }),
        _bundle,
        _signatures
    );

    if (targetSequence.enableGasRefund) {
        _executeGasRefund(
            _safeAddr,
            targetSequence.maxRefundAmount,
            targetSequence.refundRecipient,
            _gasStart
        );
    }

    //....
}
```

A normal CCTP execution flow can naturally result in nested bundle execution:

1. An executor initiates an outer bundle through `EIP712TypedDataSafeModule.executeBundle()`, recording gas snap-

shot A.

- During execution, the bundle invokes `CCTPBundleReceiver.relayWithBundle()`.
- `relayWithBundle()` receives and processes the bridged funds and subsequently calls `_executeBundleBestEffort()`.
- `_executeBundleBestEffort()` initiates a second bundle execution through a reentrant call to `executeBundle()`, recording a new gas snapshot B.
- The inner bundle executes and refunds gas based on `B - C`, where C is the remaining gas after the inner execution completes.
- Execution returns to the outer bundle, which later refunds gas based on `A - D`, where D is the gas remaining at the completion of the outer execution.

```
function relayWithBundle(
  bytes calldata message,
  bytes calldata attestation,
  address safeAddress,
  IEIP712TypedDataSafeModule.Bundle calldata bundle,
  bytes calldata signature
) external returns (
  bool relaySuccess,
  bool bundleSuccess,
  bytes memory bundleReturnData
) {
  //....

  (bundleSuccess, bundleReturnData) = _executeBundleBestEffort(safeAddress, bundle, signature);

  //....
}

function _executeBundleBestEffort(
  address safeAddress,
  IEIP712TypedDataSafeModule.Bundle calldata bundle,
  bytes calldata signature
) private returns (bool bundleSuccess, bytes memory bundleReturnData) {
  //....
  try IBravaSafeModule(module).executeBundle(safeAddress, bundle, signature) {
    bundleSuccess = true;
  } catch (bytes memory err) {
    bundleReturnData = err;
  }
}
```

However, the outer measurement already includes all gas consumed during the inner execution:

$$A - D \\ = (A - B) + (B - C) + (C - D)$$

When the inner refund is added, the total refunded amount becomes:

$$(A - D) + (B - C) \\ = (A - B) + 2(B - C) + (C - D)$$

As a result, the gas consumed by the inner bundle execution (`B - C`) is refunded twice.

Importantly, this issue does not require a malicious actor. The double-counting arises from the intended CCTP workflow where bundle execution can recursively invoke additional bundle executions. Whenever gas refunds are enabled for both executions, the protocol may reimburse executors for more gas than was actually consumed.

This can lead to systematic overpayment of executors, depletion of refund reserves, and inaccurate protocol accounting. Furthermore, if users are able to construct execution flows containing multiple nested bundle executions, the over-refund amount scales with each additional level of nesting.

Recommendations

Possible mitigations include:

- Preventing gas refunds from being processed during nested `executeBundle()` calls and only allowing the outer-most execution to perform refund settlement.
- Tracking execution depth and disabling gas accounting when an execution is already in progress.

Resolution

The development team added a `nonReentrant` guard (OpenZeppelin `ReentrancyGuard`) to `executeBundle`, making nested execution revert on-chain. Resolved in commit [653b931](#).

BRV4-02 Same-Safe Reentrancy Can Execute Later Bundles Mid-Sequence			
Assets	auth/EIP712TypedDataSafeModule.sol SequenceExecutor.sol actions/utils/GasRefundAction.sol		
Status	Resolved: See Resolution		
Rating	Severity: High	Impact: High	Likelihood: Medium

Description

`executeBundle()` can be re-entered for the same Safe while an earlier bundle is still executing, so a later nonce can observe and act on partial state before the earlier sequence completes.

`EIP712TypedDataSafeModule._executeChainSequence()` increments the Safe's sequence nonce before executing actions through the Safe. There is no per-Safe reentrancy guard around the execution window.

```
function _executeChainSequence(
    address _safeAddr,
    ITyped.Bundle callData _bundle,
    bytes callData _signatures,
    uint256 _expectedSequenceNonce,
    uint256 _gasStart,
    ITyped.ChainSequence memory targetSequence
) internal {
    // ... snip ...

    // @audit The next nonce becomes executable before this sequence completes.
    sequenceNonces[_safeAddr] = _expectedSequenceNonce + 1;

    _execThroughSafeOrRevert(
        _safeAddr,
        ISequenceExecutor.Sequence({
            name: targetSequence.sequence.name,
            callData: targetSequence.sequence.callData,
            actionIds: actionIds
        }),
        _bundle,
        _signatures
    );

    if (targetSequence.enableGasRefund) {
        _executeGasRefund(
            _safeAddr,
            targetSequence.maxRefundAmount,
            targetSequence.refundRecipient,
            _gasStart
        );
    }

    _logSequenceComplete(_safeAddr, _bundle.expiry, _expectedSequenceNonce);
}
```

The sequence executor delegate calls each registered action. A registered action, or an external callback reached by an action, can call back into the module with a valid bundle for nonce `N + 1` while nonce `N` is still in progress.

```
function executeSequence(
    Sequence calldata _currSequence,
    IEIP712TypedDataSafeModule.Bundle calldata _bundle,
    bytes calldata _signature,
    uint16 _strategyId
) public payable virtual {
    if (_currSequence.callData.length != _currSequence.actionIds.length) {
        revert Errors.EIP712TypedDataSafeModule_LengthMismatch();
    }
    for (uint256 i = 0; i < _currSequence.actionIds.length; ++i) {
        // @audit Actions execute before the outer bundle has logged completion or settled refunds.
        _executeAction(_currSequence, i, _bundle, _signature, _strategyId);
    }
}
```

This is not an unauthorised replay because the nested bundle still needs valid signatures for the next sequence nonce. The issue is that nonce order no longer guarantees completion order. A lower-privilege manager-signed bundle at nonce $N + 1$ can run in the middle of an owner-signed bundle at nonce N , potentially using assets or state that the owner-signed sequence only intended to expose temporarily before later actions completed.

Gas refunds are also affected because refund reservations are held in the module until after sequence execution. `GasRefundAction` transfers refund tokens from the Safe to the module, and `_executeGasRefund()` in `IEIP712TypedDataSafeModule` later pays from the module's aggregate balance and sweeps the remainder back to the Safe.

```
function executeAction(bytes memory _callData, uint16 /* _strategyId */) public payable override {
    // ... snip ...
    // @audit Refund funds are deposited into the shared module balance before settlement.
    IERC20(refundToken).safeTransfer(address(EIP712_MODULE), amount);
}
```

```
function _executeGasRefund(
    address safe,
    uint256 maxRefundAmount,
    uint8 refundRecipient,
    uint256 gasStart
) internal {
    IERC20 token = IERC20(usdcToken);
    // @audit A nested execution can consume deposits made by the outer execution.
    uint256 moduleDeposit = token.balanceOf(address(this));

    if (moduleDeposit == 0) return;
    // ... snip ...
}
```

A nested bundle can therefore settle against refund funds deposited by an outer bundle before the outer bundle reaches its own refund settlement. Completion logs can also be emitted in nested order, with nonce $N + 1$ completing before nonce N .

This issue has a high impact because signed sequencing can be used to protect temporary state transitions and refund balances, and same-Safe reentrancy breaks the expected atomic completion ordering. This issue has a medium likelihood because exploitation requires a reentrant action path or callback and a valid signed bundle for the next nonce, but the module itself does not prevent the pattern.

Recommendations

Add a per-Safe reentrancy guard to `executeBundle()` so the same Safe cannot start another bundle while one is already executing. The guard should clear in all revert paths by using a modifier or equivalent structured control flow.

Keep the guard per Safe rather than global so independent Safes can execute concurrently without blocking each other.

Resolution

The development team added a contract-wide `nonReentrant` guard (OpenZeppelin `ReentrancyGuard`) to `executeBundle`, preventing any re-entry into bundle execution while a bundle is in progress. Resolved in commit [653b931](#).

BRV4-03	Atomic Cross Chain CCTP Bundle Execution Can Be Griefed		
Assets	auth/AuthRegistry.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Cross-chain bundles sent via the Circle CCTP can be blocked from executing requiring the safe to resubmit the bundle for later execution.

Both `receiveCCTPAuthUpdate()` and `relayCCTPAndExecute()` rely on the same `CCTP message` and `attestation` fields, meaning that data intended for `relayCCTPAndExecute()` can also be executed by `receiveCCTPAuthUpdate()`.

An attacker can monitor the mempool listening for transactions including `relayCCTPAndExecute()` and frontrun these transactions with calls to `receiveCCTPAuthUpdate()` containing the same `message` and `attestation` inputs. As the CCTP message transmitter makes use of message nonces this will force the second call to fail as the message has already been played.

Below we have highlighted where this happens in both functions:

```

/// @notice Receives a Circle-attested CCTP V2 message carrying an auth config snapshot from the
///         source chain and copies it locally.
/// @param _message Encoded CCTP V2 message containing the burn message and optional hook data.
/// @param _attestation Circle attestation proving the message was finalized.
function receiveCCTPAuthUpdate(
    bytes calldata _message,
    bytes calldata _attestation
) external {
    (bytes32 burnSender, bytes32 mintRecipient, bytes calldata hookData) = _parseCCTPMessage(_message);

    if (!IMessageTransmitterV2(MESSAGE_TRANSMITTER).receiveMessage(_message, _attestation)) { //audit the message transmitter
        ↪ confirms the message is valid and has not been seen before.
        revert Errors.AuthRegistry_CCTPRelayFailed();
    }

    _applyHookSnapshot(hookData, burnSender, mintRecipient);
}

```

As the second call to `IMessageTransmitter.receiveMessage()` will revert, the supplied bundle will not execute:


```
// =====
// Path 3 - combined CCTP relay + bundle execution
// =====
/// @notice Mints USDC, optionally applies an auth config snapshot from `_message`'s hookData,
///         and then best-effort executes `_bundle` through the destination Safe's currently-enabled
///         Brava module.
/// @param _message Encoded CCTP V2 message containing the mint recipient and optional hook data.
/// @param _attestation Circle attestation proving the message was finalized.
/// @param _safe Destination Safe expected to receive the minted funds and execute the bundle.
/// @param _ownerAddress Owner EOA for optimistic Safe deployment. If non-zero and the Safe
///         doesn't exist yet, the relay deploys it after auth apply and before bundle execution.
///         Verified via `predictSafeAddress(ownerAddress) == _safe`. Pass `address(0)` to skip.
/// @param _bundle EIP-712 bundle forwarded to the enabled Brava module.
/// @param _signatures Signatures authorizing `_bundle` execution.
/// @return authApplied True when hook data contained and applied an auth snapshot.
/// @return bundleSuccess True when a Brava module was found and executed `_bundle` without reverting.
/// @return bundleReturnData Revert data returned by the Brava module when bundle execution fails.
function relayCCTPAndExecute(
    bytes calldata _message,
    bytes calldata _attestation,
    address _safe,
    address _ownerAddress,
    ITyped.Bundle calldata _bundle,
    bytes calldata _signatures
) external returns (bool authApplied, bool bundleSuccess, bytes memory bundleReturnData) {
    (bytes32 cctpNonce, uint32 sourceDomain) = _decodeMessageMetadata(_message);
    _assertSafeMatchesMintRecipient(_message, _safe);
    authApplied = _receiveAndMaybeApply(_message, _attestation); //@audit the CCTP message transmitter will revert inside this
    ↪ call.

    if (_ownerAddress != address(0)) {
        _optimisticDeploySafe(_safe, _ownerAddress);
    }

    address module;
    (module, bundleSuccess, bundleReturnData) = _executeBundleBestEffort(_safe, _bundle, _signatures); //@audit this bundle will
    ↪ not be executed

    LOGGER.logActionEvent(
        IActionBase.LogType.CCTP_RELAY_AND_EXECUTE,
        abi.encode(_safe, module, authApplied, bundleSuccess, sourceDomain, cctpNonce)
    );
}
```

This issue has a medium impact as the transactions that were intended to be executed atomically with any USDC mint will not occur, forcing them to be repeated. For some timing critical operations this may result in indirect user losses such as when attempting to prevent account liquidation on Aave.

This finding has a medium likelihood as frontrunning transactions by listening to the transaction mempool is non-trivial to setup and on some networks such as layer 2s is not possible, however on networks where it is technically possible it is likely to occur given an attacker would be able to economically gain from such an attack if they then trade against affected positions the Safe wallet holds on integrated modules.

Recommendations

There is a range of solutions to this finding:

- Modify the CCTP messages sent to include a flag denoting the intended function to prevent `receiveCCTPAuthUpdate()` executing CCTP messages intended for `relayCCTPAndExecute()`.
- Advise users to submit sensitive CCTP-related transactions through private mempools where pre-inclusion visibility is a concern.
- Document that CCTP-transmitted bundles should not be used for sensitive operations where chain-native signatures are more appropriate.

Resolution

The vulnerable functions `AuthRegistry.receiveCCTPAuthUpdate` and `AuthRegistry.relayCCTPAndExecute` were removed entirely as part of an `AuthRegistry` rewrite. Cross-chain auth no longer travels over CCTP; the front-running race condition is therefore no longer possible. Resolved in commits [8241f46](#) and [69f593c](#).

BRV4-04	Bundle Execution Reverts Prevent Executors From Receiving Gas Refunds		
Assets	auth/EIP712TypedDataSafeModule.sol		
Status	Closed: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Executors may lose the entire gas cost of bundle execution when a sequence fails, as bundle execution reverts before the gas refund mechanism can be reached.

During chain sequence execution, the module invokes `_execThroughSafeOrRevert()` which explicitly bubbles up any execution failure:

```
(bool ok, bytes memory returnData) = _execThroughSafe(_safeAddr, _execSeq, _bundle, _signatures);
if (!ok) {
    if (returnData.length > 0) {
        assembly {
            revert(add(returnData, 0x20), mload(returnData))
        }
    }
    revert Errors.EIP712TypedDataSafeModule_ExecutionFailed();
}
```

As a result, any failure during sequence execution causes the entire bundle execution transaction to revert. Since state changes are rolled back, any subsequent gas refund action cannot be executed, leaving the executor responsible for the full gas cost of the failed transaction.

This behaviour is particularly problematic because execution failures may occur for reasons outside of the executor's control. For example, after a bundle has been signed and submitted to an executor, a Safe owner or another authorized party may modify the Safe's state, permissions, balances, or protocol configuration in a way that causes a previously valid bundle to fail during execution. In such cases, the executor has already spent resources validating, submitting, and executing the bundle but receives no compensation.

Recommendations

Consider isolating sequence execution failures from the gas refund mechanism so that executors can still be compensated for gas spent on legitimate execution attempts.

One approach is to wrap sequence execution in a `try/catch`-style pattern (or equivalent low-level call handling) that captures failures without reverting the entire transaction. Another approach is to include the gas-limit in the bundle data as well so the amount of required gas is predefined.

Resolution

The development team have closed this issue with the following comment:

The refund is an in-sequence reservation bounded by the signed `maxRefundAmount` and atomic with the bundle. Paying a refund on a reverted bundle would decouple payment from success and let anyone holding a validly-

signed bundle revert on purpose and still drain up to `maxRefundAmount` from the Safe. Funding is kept coupled to success precisely to avoid that; executors simulate bundles before relaying, so the residual cost is operational and bounded.

BRV4-05 Gas Refund Mechanism Fails To Account For Full CCTP Relay Execution Costs			
Assets	auth/AuthRegistry.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Executors receive insufficient gas refunds when bundles are executed through `AuthRegistry.relayCCTPAndExecute()` due to gas consumption occurring before the bundle reaches the module's gas accounting boundaries.

The `relayCCTPAndExecute()` flow performs several potentially expensive operations before forwarding execution to `IBravaSafeModule.executeBundle()`:

```
authApplied = _receiveAndMaybeApply(_message, _attestation);

if (_ownerAddress != address(0)) {
    _optimisticDeploySafe(_safe, _ownerAddress);
}

(module, bundleSuccess, bundleReturnData) =
    _executeBundleBestEffort(_safe, _bundle, _signatures);
```

In particular:

- `_receiveAndMaybeApply()` calls Circle's `receiveMessage()` and processes hook data.
- `_applyHookSnapshot()` may update authorization state.
- `_optimisticDeploySafe()` may deploy a Safe when it does not yet exist.
- Module discovery is performed through `_executeBundleBestEffort()` before the bundle is forwarded.

If gas refunds are calculated using the gas accounting implemented inside the Brava module, the measurement only begins once `executeBundle()` is entered. As a result, all gas consumed by the relay path prior to module execution is excluded from refund calculations despite being paid by the executor.

Recommendations

Implement end-to-end gas accounting for relayed executions by measuring gas consumption from the entry point of `relayCCTPAndExecute()` until transaction completion, rather than only within `executeBundle()`. For example by capturing gas at the beginning of `relayCCTPAndExecute()` and passing the context into the module's refund mechanism.

Resolution

The `relayCCTPAndExecute` entry point was removed entirely. For the remaining relay path in `CCTPBundleReceiver.relayWithBundle`, a configurable overhead allowance was introduced that is added to the measured gas cost when the refund is calculated for CCTP relay calls, capped by the signed `maxRefundAmount`. Resolved in commit [686c6ff](#).

BRV4-06	Emergency Withdrawals Can Be Used To Bypass Brava Fees		
Assets	auth/EmergencyWithdrawModule.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

The emergency withdrawal path allows users to exit with their full ETH or ERC20 balance while bypassing Brava's accumulated annual fee collection.

The emergency withdraw module exists to enable Safe owners to remove assets should Brava ever be compromised, however the result of this is that a user can bypass the payment of any owed fees to Brava by utilising this module to remove the tokenised representations of liquidity such as aTokens used by Aave. Once removed, these positions can then be exited using the frontend of the underlying protocol, again, avoiding payment of fees to Brava.

```
function withdrawERC20(address safe, address token, address to, uint256 amount) external {
    _validateOwners(safe, to);

    uint256 transferAmount = amount;
    if (transferAmount == type(uint256).max) {
        transferAmount = IERC20(token).balanceOf(safe);
    }

    bytes memory data = abi.encodeCall(IERC20.transfer, (to, transferAmount));
    (bool success, bytes memory returnData) = ISafe(safe).execTransactionFromModuleReturnData(token, 0, data,
        ↳ Enum.Operation.Call);
    if (!success) {
        revert EmergencyWithdraw_TransferFailed();
    }
    if (returnData.length > 0 && !abi.decode(returnData, (bool))) {
        revert EmergencyWithdraw_TransferFailed();
    }

    emit EmergencyERC20Withdrawal(safe, token, to, transferAmount);
}
```

The testing team has rated this as a low impact issue as only fees owed to Brava are at risk and no client funds are vulnerable. Furthermore, as advised by Brava, there are offchain processes in place which would allow for direct recovery of fees. The finding has also been rated as a medium likelihood due to the low technical complexity needed to execute this finding by an end user.

Recommendations

The testing team acknowledges that eliminating this issue completely may be technically very difficult without introducing a dependency on Brava that then renders this module ineffective for its purpose. Some possible methods to adopt are:

- Rely on contractual obligations with users to enforce the payment of owed fees, making use of monitoring tools to ensure these are collected.
- Add a safe contract fuse so that once the emergency withdraw module has been used no other modules may be called from that safe. This has the benefit of preventing casual use of the emergency withdraw module but would

also increase user friction with the need to set up a new safe as well as potentially freezing any remaining assets that are not represented by ERC20 deposit tokens.

Resolution

The development team have closed this issue with the following comment:

The emergency module is deliberately built with zero dependency on any Brava-controlled contract — its sole purpose is guaranteeing owners can recover assets even if Brava is fully compromised or disappears. Adding a fee gate reintroduces exactly the dependency the module exists to remove and would let a compromised Brava ransom user funds. Additionally, onchain fees are disabled (`feeBasis == 0`); fee collection is contractual and offchain, so there is no onchain fee for this path to bypass. We rely on contractual enforcement plus monitoring, which matches the report's own first recommendation.

BRV4-07 CEI Violation In Fee Processing Enables Double-Fee Reentrancy			
Assets	actions/ActionBase.sol actions/morpho-markets/MorphoMarketsSupply.sol actions/morpho-markets/MorphoMarketsWithdraw.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The fee processing functions in both `ActionBase._processFee()` and `MorphoMarkets._processMorphoFee()` update the fee timestamp state after performing external token transfers, violating the checks-effects-interactions pattern. A callback-capable fee token or malicious fee recipient could re-enter before the timestamp is updated and trigger duplicate fee collection.

In `ActionBase._processFee()`:

```
if (fee > 0) {
    vault.safeTransfer(ADMIN_VAULT.feeConfig().recipient, fee);
}
ADMIN_VAULT.setFeeTimestamp(_pool); // @audit state update after external transfer – CEI violation
```

Similarly in `MorphoMarketsSupply._processMorphoFee()`:

```
if (fee > 0) {
    (uint256 actualFee,) = _morpho.withdraw(_marketParams, fee, 0, address(this), address(this));
    IERC20(_marketParams.loanToken).safeTransfer(ADMIN_VAULT.feeConfig().recipient, actualFee);
    feeInTokens = actualFee;
}
// @audit setFeeTimestamp called after both external calls
ADMIN_VAULT.setFeeTimestamp(feeKey);
```

On re-entry, `ADMIN_VAULT.getLastFeeTimestamp()` still returns the old timestamp, so the same-block guard does not trigger and fees are calculated and transferred again for the same time window.

This issue has a medium impact as it enables double-fee extraction from user positions, though extracted value is routed to the protocol fee recipient rather than an arbitrary attacker. This issue has a low likelihood as it requires the fee token to implement transfer hooks (ERC-777) or the fee recipient to be a malicious contract, both of which are protocol-controlled configurations.

Recommendations

Move `ADMIN_VAULT.setFeeTimestamp()` to before any external calls in both `_processFee()` and `_processMorphoFee()`, following the checks-effects-interactions pattern.

Resolution

`setFeeTimestamp` now runs before the external fee transfer in all three fee sites: `ActionBase._processFee` and the bespoke `_processMorphoFee` in `MorphoMarketsSupply` and `MorphoMarketsWithdraw`. Resolved in commit [07d4453](#).

BRV4-08	proposeToken Overwrite Resets Delay Timer		
Assets	auth/TokenRegistry.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

`TokenRegistry.proposeToken()` does not check whether a proposal already exists for the given token, allowing a proposer to overwrite active proposals and reset the delay timer.

```
function proposeToken(address _token) external onlyRole(Roles.TRANSACTION_PROPOSER_ROLE) {
    require(_token != address(0), Errors.InvalidInput("TokenRegistry", "proposeToken"));
    require(!approvedTokens[_token], Errors.AdminVault_TransactionAlreadyApproved());

    // @audit Missing: require(tokenProposals[_token] == 0, ...)
    tokenProposals[_token] = _getDelayTimestamp();
    LOGGER.logAdminVaultEvent(106, abi.encode(_token));
}
```

In contrast, both `cancelTokenProposal()` and `approveToken()` explicitly require `tokenProposals[_token] != 0`, indicating the codebase distinguishes between proposed and unproposed states.

A compromised `TRANSACTION_PROPOSER_ROLE` key can exploit this to defeat cancellation monitoring: after a cancel bot successfully removes a malicious proposal, the attacker immediately re-proposes the same token, restarting the delay. If monitoring does not re-watch for the same token after cancellation, the proposal can expire uncontested.

Additionally, if the protocol delay is reduced between proposals, the attacker can re-propose to replace the original long-delay timestamp with a shorter one, circumventing the original security window.

This issue has a medium impact as it undermines the timelock security model for token registration. This issue has a low likelihood as it requires a compromised proposer role key.

Recommendations

Add `require(tokenProposals[_token] == 0)` at the start of `proposeToken()`, consistent with the role proposal pattern. If re-proposing is a desired administrative action, provide a separate function with distinct event emission.

Resolution

The development team have closed this issue with the following comment:

We agree the guard would be a hardening, but the exposure does not justify replacing a deployed core singleton. Exploitation requires a compromised `TRANSACTION_PROPOSER_ROLE` key, and even then an overwrite can only push a pending proposal's executable time later — it can never shorten the delay or approve a token early. The practical impact is a griefing reset of a pending timelock, which is already covered operationally: every proposal emits a monitored `AdminVaultEvent`, and `TRANSACTION_CANCELER_ROLE` can cancel unexpected proposals within the delay window.

BRV4-09 Emergency ETH Withdrawals May Revert For Smart Contract Owners			
Assets	auth/EmergencyWithdrawModule.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

The Emergency Withdraw Module can send ETH to the Safe owner however not all smart contracts are capable of accepting direct ETH transfers and may revert on such attempts preventing the reclamation of ETH in the Safe Brava account.

This can occur because smart contracts must include a payable `fallback()` function or a `receive()` function in order to receive ETH with a direct empty function call. Below is the affected function:

```
function withdrawETH(address safe, address to, uint256 amount) external {
    _validateOwners(safe, to);

    uint256 transferAmount = amount;
    if (transferAmount == type(uint256).max) {
        transferAmount = safe.balance;
    }

    bool success = ISafe(safe).execTransactionFromModule(to, transferAmount, "", Enum.Operation.Call); //@audit this call will send
    ↳ ETH which will fail if the to address is a smart contract that cannot accept ETH
    if (!success) {
        revert EmergencyWithdraw_TransferFailed();
    }

    emit EmergencyETHWithdrawal(safe, to, transferAmount);
}
```

This issue has a medium impact as in the event that the emergency withdraw module is required it is reasonable to assume other modules are no longer accessible meaning the stuck ETH cannot be converted to another asset via a different module action.

However, this should only occur if the owner address is a smart contract itself and also does not declare any functions that can accept ETH deposits such as `receive()` or a payable `fallback()` function.

Recommendations

Add documentation to ensure end users of the Safe wallet know of this limitation. Alternatively, an additional function to wrap ETH into Wrapped ETH could be added to the emergency withdraw module which would enable it to then be withdrawn via the ERC20 route.

Resolution

The development team have closed this issue with the following comment:

In Brava's model, owners are EOAs by construction. Operating a Brava Safe requires producing EIP-712 ECDSA signatures – at minimum the Owner-signed AuthBundle that configures the Safe's managers and co-signers –

which a smart-contract account cannot produce as a recoverable secp256k1 signer. A contract that cannot sign could never have been configured as a functioning Brava owner, so the “contract owner that rejects ETH” precondition does not arise for any Safe actually operated through Brava. Separately, we hold WETH (an ERC-20) rather than native ETH. The EOA-owner requirement is documented.

BRV4-10	Cosigner Owners Can Make Signing Threshold Unreachable		
Assets	auth/AuthRegistry.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Safe wallets with owners set as cosigners can fail to meet signing thresholds making it impossible for managers to approve actions.

When a new authorisation configuration is set the threshold value is compared to the number of cosigners to ensure that the threshold is achievable. The code handling this is seen below:

```
function _validateSnapshot(
    uint256 _newVersion,
    address[] memory _newManagers,
    address[] memory _newCoSigners,
    uint256 _managerCoSignThreshold
) private pure {
    // snipped...
    if (_managerCoSignThreshold > _newCoSigners.length) { //@audit this condition ignores the possibility of an owner being a
        ↪ cosigner
        revert Errors.AuthRegistry_InvalidThreshold(_managerCoSignThreshold, _newCoSigners.length);
    }

    _assertValidMembers(_newManagers, _newCoSigners);
}
```

There is no validation to ensure a owner is not listed as a cosigner. It is important to note that Brava are already aware of this design as evidenced by the following code comment:

```
/// Note: this function does NOT check whether manager/cosigner addresses are also Safe
/// owners. If a Safe owner is registered as a manager or cosigner, the EIP-712 module's
/// signer classification (isOwner > isCoSigner > isManager) will classify them as Owner,
/// silently reducing the effective manager/cosigner count.
```

This reduces the achievable `coSignCount` by one which can prevent strict thresholds such as 5 of 5 cosigner thresholds being reached. This is due to the classification of bundle signers where the owner address cannot be treated as a cosigner even if listed as one:

```

/// @notice Classifies each recovered signer against the Safe's owner list and AuthRegistry.
///         The Safe MUST be deployed before this is called - classification queries
///         IOwnerManager.isOnChain.
///         Owner classification is strictly highest priority. If the same address is both a Safe
///         owner and a registered co-signer/manager, it is always classified as Owner.
/// @param _safeAddr The deployed Safe whose owner list is used for classification
/// @param _signers Recovered signer addresses (from _recoverSigners)
/// @return hasOwner True if at least one signer is a Safe owner
/// @return hasManager True if at least one signer is a registered manager
/// @return managerAddr The manager address (address(0) if no manager)
/// @return coSignCount Number of co-signer signatures (does NOT include owner or manager)
function _classifySigners(
    address _safeAddr,
    address[] memory _signers
) internal view returns (
    bool hasOwner,
    bool hasManager,
    address managerAddr,
    uint256 coSignCount
) {
    for (uint256 i; i < _signers.length; ++i) {
        address signer = _signers[i];
        if (IOwnerManager(_safeAddr).isOwner(signer)) {
            hasOwner = true;
        } else if (authRegistry.isCoSigner(_safeAddr, signer)) { // @audit this clause will not trigger when a cosigner is also an
            // owner reducing the cosign count.
            ++coSignCount;
        } else if (authRegistry.isManager(_safeAddr, signer)) {
            if (hasManager) {
                revert Errors.EIP712TypedDataSafeModule_MultipleManagers();
            }
            hasManager = true;
            managerAddr = signer;
        } else {
            revert Errors.EIP712TypedDataSafeModule_SignerNotAuthorised(signer);
        }
    }
}

```

Note this issue has been rated as having a low impact and likelihood as the owner can submit a new authorisation configuration removing themselves as a cosigner and because Brava have made it clear that owners are not intended to be managers or cosigners.

Recommendations

Add validation to authorisation configuration changes that prevent owners being set as cosigners or add explicit documentation to ensure end users do not set up their safes with owners as cosigners.

Resolution

The development team have closed this issue with the following comment:

The Owner is a hands-off custodian, not a day-to-day co-signer; the signer classification precedence (owner > co-signer > manager) is deliberate and documented in-code. An owner who wishes to co-sign operationally should use a separate account. The supporting tooling is already in place: the SDK flags configurations that list an owner as a co-signer at auth-config construction time, and the operator-facing documentation covers the classification precedence.

BRV4-11	Raw Revert Bubbling Can Spoof Module Errors		
Assets	auth/EIP712TypedDataSafeModule.sol SequenceExecutor.sol Errors.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Medium	Likelihood: Low

Description

When a sequence fails, the module returns the inner revert bytes as its own top-level revert. As a result, an action or downstream call can make the failed bundle appear to have failed with a native `EIP712TypedDataSafeModule` error, at least to offchain systems that identify errors only by their 4-byte selector.

`EIP712TypedDataSafeModule._execThroughSafeOrRevert()` executes the sequence through the Safe. If that execution fails and returns revert data, the module reverts with those bytes unchanged. It does not wrap the failure in a module error that says "this came from sequence execution" or otherwise preserves the boundary between module validation and action execution.

```
function _execThroughSafeOrRevert(
    address _safeAddr,
    ISequenceExecutor.Sequence memory _execSeq,
    ITyped.Bundle calldata _bundle,
    bytes memory _signatures
) internal {
    (bool ok, bytes memory returnData) = _execThroughSafe(_safeAddr, _execSeq, _bundle, _signatures);
    if (!ok) {
        if (returnData.length > 0) {
            // @audit The top-level revert payload is exactly the inner payload.
            assembly {
                revert(add(returnData, 0x20), mload(returnData))
            }
        }
        revert Errors.EIP712TypedDataSafeModule_ExecutionFailed();
    }
}
```

Those bytes can come from an action executed by `SequenceExecutor`. The executor uses `delegatecall` for each action and, if the action fails, also propagates the action's revert data unchanged.

```
function _executeActionStandard(address _actionAddress, bytes memory _fullCallData) internal {
    // ... snip ...
    assembly {
        let succeeded := delegatecall(sub(gas(), 5000), _actionAddress, add(_fullCallData, 0x20), mload(_fullCallData), 0, 0)
        if eq(succeeded, 0) {
            // @audit Action revert bytes are propagated unchanged.
            returndatacopy(0, 0, returndatasize())
            revert(0, returndatasize())
        }
    }
}
```

Custom errors are ABI-encoded as a 4-byte selector followed by encoded arguments. Any contract in this action path can intentionally or accidentally return bytes with the same selector and argument layout as a module error, such as `EIP712TypedDataSafeModule.ActionNotFound(bytes4)` or `EIP712TypedDataSafeModule.InsufficientCoSignatures(uint256,uint256)`. For example, a registered action, a protocol called by that action, or an incorrectly registered action address could emit such bytes. Because the module forwards them unchanged, offchain monitors that treat the top-level selector as authoritative can misclassify where the failure came from and why it occurred.

This issue has a medium impact because misleading failure attribution can affect monitoring, alerting, incident response, and automated relayer behaviour. This issue has a low likelihood because the misleading revert must originate from a registered action path, a downstream protocol called by that action, or an incorrectly registered action.

Recommendations

Wrap failed sequence execution in a module-specific error that preserves the inner bytes without making them the top-level selector. For example, add an error such as `EIP712TypedDataSafeModule_ExecutionReverted(bytes reason)` and revert with that from `_execThroughSafeOrRevert()`.

Add action context in `SequenceExecutor` failures where practical, such as the action index and action ID, so offchain tooling can distinguish module validation failures from action execution failures.

Resolution

`SequenceExecutor` now wraps any action failure in `SequenceExecutor_ActionReverted(uint256 actionIndex, bytes4 actionId, bytes reason)`, so the top-level revert selector is owned by `SequenceExecutor` and cannot be spoofed by an action. The original revert bytes are preserved in `reason`. Resolved in commit [e78b30b](#).

BRV4-12 Bundle Signatures Do Not Bind Auth Config Version			
Assets	auth/EIP712TypedDataSafeModule.sol libraries/EIP712TypedDataLib.sol auth/AuthRegistry.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Unexpired bundle signatures can be executed under a different auth configuration from the one that existed when the signatures were collected, which can violate co-signer intent after a Safe's manager set, co-signer set, threshold, or manager restrictions change.

`EIP712TypedDataSafeModule.executeBundle()` hashes only the bundle contents and the Safe address. It then classifies recovered signers against the current `AuthRegistry` state at execution time. The bundle digest does not commit to the registry version or to any hash of the active auth configuration.

```
function executeBundle(
    address _safeAddr,
    ITyped.Bundle calldata _bundle,
    bytes calldata _signatures
) external onlyInitialized {
    // ... snip ...

    bytes32 digest = EIP712TypedDataLib.hashBundleForSigning(domainName, domainVersion, _safeAddr, _bundle);

    uint256 expectedSequenceNonce = sequenceNonces[_safeAddr];

    address[] memory signers = _recoverSigners(digest, _signatures);

    // ... snip ...

    // @audit Signers are checked against the live registry state, not the state at signing time.
    (bool hasOwner, bool hasManager, address managerAddr, uint256 coSignCount) = _classifySigners(_safeAddr, signers);

    _verifyThresholds(_safeAddr, hasOwner, hasManager, coSignCount);
}
```

The EIP-712 type used for bundle signing includes the sequence nonce and sequence data, but does not include the auth registry version.

```
string private constant BUNDLE_TYPE = "Bundle(uint256 expiry,ChainSequence[] sequences)ActionDefinition(string protocolName,uint8
↳ actionType)ChainSequence(uint256 chainId,uint256 sequenceNonce,bool deploySafe,bool enableGasRefund,uint256
↳ maxRefundAmount,uint8 refundRecipient,Sequence sequence)Sequence(string name,ActionDefinition[] actions,bytes4[]
↳ actionIds,bytes[] callData)";

// ... snip ...

function hashBundle(IEip712TypedDataSafeModule.Bundle memory bundle) public pure returns (bytes32) {
    // ... snip ...
    return keccak256(abi.encode(
        BUNDLE_TYPEHASH,
        bundle.expiry,
        keccak256(abi.encodePacked(chainSequenceHashes))
    ));
}
```

For example, a future manager can collect co-signer signatures for a bundle while they are not yet an authorised manager. If a Safe owner later applies a registry update that makes that address a manager before the bundle expires, the

same signatures can satisfy the live threshold and execute the bundle. The sequence nonce prevents re-execution of a consumed bundle, but it does not prevent first execution under stale authorisation assumptions.

This issue has a low impact because execution still requires valid signatures and an auth configuration change authorised by a Safe owner. The main risk is that co-signers approve a digest under one policy but the digest is later interpreted under another policy. This issue has a low likelihood because it requires timing between signature collection and expiry, plus a relevant auth configuration change before execution.

Recommendations

Bind bundle signatures to the auth configuration used during signing. For example, include the current `AuthRegistry` version, or a hash of the relevant auth snapshot, in each signed chain sequence and require it to match the live registry before execution.

Alternatively, invalidate outstanding sequence bundles whenever the auth configuration changes by maintaining a per-Safe auth epoch in the module and incrementing it from the registry update path.

Resolution

The development team have closed this issue with the following comment:

Classifying signers against the live registry state is the intended model, not a gap: the only policy that matters is the one in force at execution time, and a bundle can only execute if every signature satisfies that policy at that moment. The scenario the finding highlights: a manager's pre-collected signatures becoming valid once auth is granted: is a legitimate workflow, not an escalation: a manager may compile and sign a plan ahead of approval, and that bundle is inert until the owner's signed auth update lands. The grant is the authorisation act; nothing executes that the current policy does not permit, and revoking a signer immediately invalidates their in-flight bundles at the execution check. Binding the auth version into the signed digest would break this intended pre-planning workflow and invalidate every in-flight signature on any auth change, in exchange for removing a property we consider correct.

BRV4-13	Unbounded Auth Version Can Block Future Updates		
Assets	auth/AuthRegistry.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

A Safe owner can set the auth registry version to `type(uint256).max`, after which no higher version exists and all future auth configuration changes for that Safe become impossible through normal registry updates.

`AuthRegistry._apply()` accepts any non-zero version that is greater than the current version. If the new version equals the current version, the update is allowed only when the supplied config exactly matches the stored config. This makes `type(uint256).max` a terminal version for any non-identical future snapshot.

```
function _apply(
    address _safe,
    uint256 _newVersion,
    address[] memory _newManagers,
    address[] memory _newCoSigners,
    uint256 _managerCoSignThreshold,
    IAuthRegistry.ManagerRestriction[] memory _restrictions
) internal {
    _validateSnapshot(_newVersion, _newManagers, _newCoSigners, _managerCoSignThreshold);

    AuthConfig storage state = _state[_safe];
    uint256 currentVersion = state.version;

    if (_newVersion < currentVersion) {
        revert Errors.AuthRegistry_StateVersion(_newVersion, currentVersion);
    }
    if (_newVersion == currentVersion) {
        if (!_configMatches(state, _safe, _newManagers, _newCoSigners, _managerCoSignThreshold, _restrictions)) {
            revert Errors.AuthRegistry_ConflictingConfig(_newVersion);
        }
        return;
    }

    // ... snip ...

    // @audit `type(uint256).max` is accepted and leaves no possible higher version.
    state.version = _newVersion;
}
```

The snapshot validator rejects only version zero and does not cap version jumps.

```
function _validateSnapshot(
    uint256 _newVersion,
    address[] memory _newManagers,
    address[] memory _newCoSigners,
    uint256 _managerCoSignThreshold
) private pure {
    // @audit Any non-zero version is valid, including type(uint256).max.
    if (_newVersion == 0) revert Errors.AuthRegistry_VersionZero();
    // ... snip ...
}
```

This issue has a low impact because the update must be signed by a Safe owner, and Safe owners retain their normal Safe-level authority. However, it can permanently disable the manager/co-signer authorisation layer for that Safe unless a migration or contract-level recovery path exists. This issue has a low likelihood because it requires an owner mistake, malicious owner action, or compromised owner signature.

Recommendations

Limit auth version increments to a bounded range above the current version, or require the next version to be exactly `currentVersion + 1`. This preserves monotonic replay protection while preventing a single update from consuming the entire version space.

Resolution

The registry now bounds the forward version jump: an update may exceed the current version by at most 10, reverting `AuthRegistry_VersionJumpTooLarge` otherwise. This preserves the multi-chain replay model while preventing version-space exhaustion. Resolved in commit [69f593c](#).

BRV4-14	Stale proposedDelay Can Override A Later Delay Increase		
Assets	AccessControlDelayed.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`AccessControlDelayed.changeDelay()` can leave a stale `proposedDelay` value behind after a previous delay-reduction lock has expired. If the owner later increases the delay, the next call to `_getDelayTimestamp()` can silently overwrite that increase with the stale lower delay.

The function only clears pending delay-reduction state while `delayReductionLockTime` is still in the future. Once the lock time has passed, the cleanup block is skipped. The later increase branch then sets `delay = _newDelay` without clearing any non-zero `proposedDelay` left from the earlier reduction.

```
function changeDelay(uint256 _newDelay) public onlyRole(OWNER_ROLE) {
    // (_newDelay must be different from delay OR there must be a proposal to cancel)
    // AND _newDelay must not more than 5 days (to avoid costly mistakes)
    require(
        (_newDelay != delay || proposedDelay == 0) && _newDelay <= MAX_DELAY,
        Errors.AccessControlDelayed_InvalidDelay()
    );

    if (block.timestamp < delayReductionLockTime) {
        // The delay must already have been reduced because delayReductionLockTime is in the future
        // We can't have set the delay to proposedDelay yet, so we can just delete it
        delete delayReductionLockTime;
        delete proposedDelay;
    }
    LOGGER.logAdminVaultEvent(400, abi.encode(delay, _newDelay));
    if (_newDelay >= delay) {
        // New delay is longer, just set it
        delay = _newDelay;
    } else {
        // New delay is shorter, enforce old delay until it is met
        delayReductionLockTime = block.timestamp + delay;
        proposedDelay = _newDelay;
    }
}
```

Later, `_getDelayTimestamp()` applies any non-zero `proposedDelay` as a side effect once `delayReductionLockTime` is no longer in the future.

```
function _getDelayTimestamp() internal returns (uint256) {
    if (block.timestamp < delayReductionLockTime) {
        // We haven't reached the lock time yet,
        // We must wait until the greater of the lock time, or now + proposedDelay
        uint256 proposedDelayTime = block.timestamp + proposedDelay;
        return proposedDelayTime > delayReductionLockTime ? proposedDelayTime : delayReductionLockTime;
    }
    // We have reached the lock time, we may set the delay to the proposed delay
    if (proposedDelay != 0) {
        delay = proposedDelay;
        delete proposedDelay;
    }
    return block.timestamp + delay;
}
```

As a result, an owner may increase the delay, for example from 2 days to 5 days, but the next proposal call can apply an old stale `proposedDelay`, such as 1 day, and reduce the stored delay again. This can cause proposals created after the

owner's increase to receive a shorter timelock than intended. The impact is limited because the sequence requires an expired prior delay reduction to remain unapplied, but the resulting state change can be triggered by any later proposal path that calls `_getDelayTimestamp()`.

Recommendations

Clear any pending or stale delay-reduction state whenever the owner sets a delay that is greater than or equal to the current delay. Alternatively, make `changeDelay()` apply or clear an expired `proposedDelay` before evaluating the requested new delay.

Resolution

The development team have closed this issue with the following comment:

We agree the state machine has a stale-value edge, but it is reachable only by a specific sequence performed entirely by our own `OWNER_ROLE`: reduce the delay, let the reduction lock expire with no intervening proposal activity (any proposal materialises the pending value), then increase the delay. No external party can trigger it. We handle it in the delay-change runbook instead of a redeploy: after any `changeDelay`, the operator verifies `proposedDelay == 0` and `delayReductionLockTime == 0` onchain (both are public), and if a stale reduction is present it is flushed with a follow-up `changeDelay` before the change is signed off. Delay changes are rare, owner-gated, and emit a monitored event.

BRV4-15 Accumulating ERC20 Allowance In BuyCover			
Assets	actions/nexus-mutual/BuyCover.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Each cover purchase can leave unused ERC20 allowance for the Nexus Mutual `COVER_BROKER`. Over time, these residual allowances can accumulate, increasing the amount of funds the broker could transfer if it were compromised or upgraded maliciously.

In `BuyCover._buyCover()`, the code grants additive allowance via `safeIncreaseAllowance` before invoking `COVER_BROKER.buyCover()`. The actual premium paid can be less than `maxPremiumInAsset`. When that happens, the unused portion of the allowance remains after the purchase and the next call adds a new allowance on top of it.

```
paymentAsset.safeIncreaseAllowance(address(COVER_BROKER), params.maxPremiumInAsset);  
  
coverId = COVER_BROKER.buyCover(params, poolAllocationRequests);  
  
uint256 balanceAfter = paymentAsset.balanceOf(address(this));  
  
premiumPaid = balanceBefore - balanceAfter;  
  
// @audit residual allowance (maxPremiumInAsset - premiumPaid) persists
```

This issue has a low impact as exploitation requires the hardcoded Nexus Mutual broker contract to be compromised or upgraded maliciously. This issue has a low likelihood as it depends on broker compromise and is not directly attacker-exploitable.

Recommendations

Reset the allowance to zero after the broker call returns, using `forceApprove(address(COVER_BROKER), 0)`. This mirrors the set-and-reset pattern already used in `ZeroExSwap`.

Resolution

`BuyCover` now uses a strict set-then-clear pattern: `forceApprove` to the premium ceiling before the purchase and `forceApprove(0)` after, so no residual allowance accumulates. Resolved in commit [078f467](#).

BRV4-16	Missing USDC Token Validation In CCTPBridgeSend		
Assets	actions/cctp/CCTPBridgeSend.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

CCTPBridgeSend() does not validate that the user-supplied `params.usdcToken` matches the expected USDC address, allowing any CCTP-registered token to be bridged without on-chain rejection.

The only check on `params.usdcToken` is a zero-address guard. Because CCTP V2's `TOKEN_MESSENGER` supports multiple registered tokens (e.g. USDC and EURC), an authorised manager could supply a different CCTP-registered token and bridge an unintended asset.

```
if (
  params.usdcToken == address(0) ||
  params.amount == 0 ||
  params.destinationCaller == bytes32(0)
) revert Errors.InvalidInput("CCTPBridgeSend", "executeAction");
// @audit no check that params.usdcToken is actually USDC
```

This issue has a low impact as the wrong asset is bridged rather than lost, and only authorised managers can trigger the action. This issue has a low likelihood as it requires an authorised manager to supply an incorrect but valid CCTP-registered token address.

Recommendations

Store the expected USDC address as an immutable address and validate `params.usdcToken == USDC_TOKEN` in `executeAction()`.

Resolution

CCTPBridgeSend now resolves the expected USDC address from AdminVault per-chain config and reverts CCTPBridgeSend_UnexpectedToken unless `params.usdcToken` matches it exactly. Resolved in commit [8241f46](#).

BRV4-17 Swap Contracts Omit tokenIn Registry Validation			
Assets	actions/swap/ZeroExSwap.sol actions/swap/ParaswapSwap.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Both `ZeroExSwap` and `ParaswapSwap` validate `tokenOut` against `TOKEN_REGISTRY` but never check `tokenIn`, allowing authorised swaps to spend arbitrary unapproved tokens.

```
require(
  TOKEN_REGISTRY.isApprovedToken(params.tokenOut),
  Errors.ZeroEx__TokenNotApproved(params.tokenOut)
);
// @audit params.tokenIn is never checked against TOKEN_REGISTRY
```

This creates a policy bypass: the token registry gates what the Safe can receive from swaps but not what it can spend. Unapproved, non-standard, or fee-on-transfer tokens can be supplied as input without on-chain rejection.

This issue has a low impact as it bypasses registry policy for the input side only and requires authorised execution. This issue has a low likelihood as it requires an authorised operator to deliberately supply an unregistered token.

Recommendations

Add a `TOKEN_REGISTRY.isApprovedToken(params.tokenIn)` check in both swap contracts, symmetric with the existing `tokenOut` validation.

Resolution

Both `ZeroExSwap` and `ParaswapSwap` now require `params.tokenIn` to be an approved token in the token registry, symmetric with the existing `tokenOut` check. Resolved in commits [40151a1](#) (`ZeroExSwap`) and [2aac471](#) (`ParaswapSwap`).

BRV4-18 Proposal Overwrite Resets Timelock In proposeAction And proposePool			
Assets	auth/AccessControlDelayed.sol auth/AdminVault.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Both `AdminVault.proposeAction()` and `AdminVault.proposePool()` unconditionally overwrite the proposal timestamp without checking whether a proposal already exists, allowing a proposer to indefinitely reset the delay timer and prevent execution.

In contrast, `AccessControlDelayed.proposeRole()` explicitly prevents this:

```
bytes32 proposalId = keccak256(abi.encodePacked(role, account));
require(proposedRoles[proposalId] == 0, Errors.AdminVault_AlreadyProposed());
proposedRoles[proposalId] = _getDelayTimestamp();
```

The action and pool proposal functions lack this guard:

```
bytes32 proposalId = keccak256(abi.encodePacked(_actionId, _actionAddress));
// @audit no AlreadyProposed guard - overwrites existing proposal timestamp
actionProposals[proposalId] = _getDelayTimestamp();
```

If the proposer and executor roles are held by different parties (the intended separation of duties), a malicious or compromised proposer can repeatedly re-submit the same proposal each time its delay is about to expire, indefinitely blocking the executor from calling `addAction()` or `addPool()`.

This issue has a low impact as it creates a denial-of-service on governance operations rather than a direct fund-loss path. This issue has a low likelihood as it requires a compromised or malicious proposer role holder.

Recommendations

Add a pre-existing proposal check to both functions, mirroring `proposeRole()`: add `require(actionProposals[proposalId] == 0, Errors.AdminVault_AlreadyProposed())` in `proposeAction()`, and the equivalent for pool proposals.

Resolution

The development team have closed this issue with the following comment:

An overwrite requires the privileged `POOL_PROPOSER_ROLE` or `ACTION_PROPOSER_ROLE` and can only move a pending proposal's executable time later, never earlier — the delay itself cannot be bypassed. The residual is timelock-reset griefing by a compromised proposer, which the canceler roles and proposal-event monitoring already cover. Replacing the deployed `AdminVault` is not warranted for this finding.

BRV4-19	AcrossV3Supply Logs Zero sharesAfter On Fee-Only Call		
Assets	actions/across-v3/AcrossV3Supply.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

When `AcrossV3Supply.executeAction()` is called with `amount=0` (a fee-only operation), the `sharesAfter` return variable is never assigned and defaults to zero, causing the emitted `BALANCE_UPDATE` event to incorrectly record the LP position as fully liquidated.

```
// Get initial balance
sharesBefore = IERC20(lpToken).balanceOf(address(this));

feeInTokens = _processFee(lpToken, _inputData.feeBasis, lpToken);

// If we have an amount to deposit, do that
if (_inputData.amount != 0) {

    // ... deposit logic ...

    // Check received shares meet minimum
    sharesAfter = IERC20(lpToken).balanceOf(address(this));
    uint256 sharesReceived = sharesAfter + feeInTokens - sharesBefore;
    require(
        sharesReceived >= _inputData.minSharesReceived,
        Errors.Action_InsufficientSharesReceived(
            lpToken,
            protocolName(),
            uint8(actionType()),
            sharesReceived,
            _inputData.minSharesReceived
        )
    );
}
// @audit sharesAfter remains 0 when amount == 0
```

Off-chain systems that reconstruct position state from `BALANCE_UPDATE` events will incorrectly interpret the position as closed after any fee-only call.

This issue has a low impact as no on-chain state is corrupted, only event data is incorrect. This issue has a low likelihood as it only occurs during fee-only operations with `amount=0`.

Recommendations

Assign `sharesAfter` unconditionally after fee processing, before the conditional deposit block.

Resolution

`sharesAfter` is now recorded unconditionally after fee processing, so a fee-only call (`amount == 0`) reports the real LP balance instead of zero. Resolved in commit [e1dd4b2](#).

BRV4-20	Accumulating Token Allowance In ParaswapSwap		
Assets	actions/swap/ParaswapSwap.sol actions/swap/ZeroExSwap.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

ParaswapSwap uses `safeIncreaseAllowance` before each swap but never resets the allowance afterward, causing residual approval to accumulate on the `AUGUSTUS_ROUTER` across successive operations.

```
// @audit additive allowance – residual accumulates if router does not consume full amount
tokenIn.safeIncreaseAllowance(address(AUGUSTUS_ROUTER), _params.fromAmount);
```

By contrast, `ZeroExSwap` explicitly uses a set-and-reset pattern with `forceApprove` followed by a zero reset:

```
// Use strict set-and-reset because ox routes can be exact-out
tokenIn.forceApprove(allowanceTarget, _params.fromAmount);
// ... swap ...
tokenIn.forceApprove(allowanceTarget, 0);
```

If the Paraswap router proxy is compromised or upgraded maliciously, the inflated allowance provides a larger attack surface for draining tokens from the action context.

This issue has a low impact as exploitation requires the trusted router to be compromised. This issue has a low likelihood as it depends on router compromise combined with accumulated residual allowance.

Recommendations

Apply the same set-and-reset pattern used in `ZeroExSwap`: use `forceApprove` before the swap and reset to zero after the swap completes.

Resolution

`ParaswapSwap` now sets the router allowance with `forceApprove(amount)` before the swap and clears it with `forceApprove(0)` after, so no residual allowance accumulates. Resolved in commit [2aac471](#).

BRV4-21	Inverted proposedDelay Guard In changeDelay		
Assets	auth/AccessControlDelayed.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The `changeDelay()` function uses an inverted condition that prevents exact cancellation of a pending delay reduction by re-submitting the current delay.

The comment states the intent: `_newDelay` must differ from `delay` OR there must be a pending proposal to cancel. However, the test in the code, `proposedDelay == 0`, tests that there is **no** proposal to cancel:

```
// (_newDelay must be different from delay OR there must be a proposal to cancel)
// AND _newDelay must not more than 5 days (to avoid costly mistakes)
require(
  // @audit proposedDelay == 0 should be proposedDelay != 0
  (_newDelay != delay || proposedDelay == 0) && _newDelay <= MAX_DELAY,
  Errors.AccessControlDelayed_InvalidDelay()
);
```

Whenever a `proposedDelay` is outstanding, calling `changeDelay(delay)` to cancel the pending reduction reverts because `_newDelay == delay && proposedDelay != 0` makes the predicate false. If the current delay is below `MAX_DELAY`, the owner can work around this by submitting a slightly larger delay, such as `delay + 1`. This clears the pending proposal and then applies the larger delay immediately.

The edge case is most restrictive when `delay == MAX_DELAY`. In that state there is no larger permitted value, because `MAX_DELAY` is the ceiling. The owner can still replace the pending reduction with a near-maximum value such as `MAX_DELAY - 1`, but cannot cancel it while keeping the delay exactly at `MAX_DELAY`.

This issue has a low impact as a workaround exists (setting delay to one second less than maximum). This issue has a low likelihood because the exact-cancellation path is only materially constrained when operating at the maximum delay value.

Recommendations

Change `proposedDelay == 0` to `proposedDelay != 0` in the require guard to match the documented intent.

Resolution

The development team have closed this issue with the following comment:

The only operation the inverted condition blocks — cancelling a pending reduction by re-asserting the exact current delay — has a straightforward owner workaround: call `changeDelay` with a marginally different value, which passes the guard and clears the unwanted pending reduction, then restore the intended value once the lock passes. The guard never blocks increasing the delay or installing a replacement value, no funds or permissions are affected, and only `OWNER_ROLE` can reach the code path. The workaround is documented in the delay-change runbook.

BRV4-22	logAdminVaultEvent Lacks Caller Verification		
Assets	Logger.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`Logger.logAdminVaultEvent()` is declared `public` with no access control, allowing any external account to emit forged `AdminVaultEvent` entries from the canonical `Logger` contract.

```
// @audit no access control - any caller can emit governance events
function logAdminVaultEvent(uint256 _logId, bytes memory _data) public {
    emit AdminVaultEvent(_logId, _data);
}
```

The `AdminVaultEvent` encoding uses a structured `logId` namespace (1XX=Proposal, 2XX=Grant, 3XX=Cancel, 4XX=Removal) that off-chain systems use to reconstruct permission history. Unlike `logActionEvent()` which includes `msg.sender` in the emitted event, `AdminVaultEvent` contains no authenticated caller field, making forged events indistinguishable from legitimate ones.

This issue has a low impact as on-chain state is unaffected; only off-chain monitoring and dashboards are affected. This issue has a low likelihood as exploitation requires an attacker to identify and target monitoring infrastructure relying on these events.

Recommendations

Add caller authentication to `logAdminVaultEvent()`, or at minimum include `msg.sender` in the emitted event so off-chain consumers can filter by expected callers.

Resolution

`Logger.logAdminVaultEvent` now stamps `msg.sender` on `AdminVaultEvent(address caller, uint256 logId, bytes data)`, mirroring the existing `ActionEvent` shape, so offchain consumers can establish trust by filtering on `caller`. Resolved in commit [db7ba77](#).

BRV4-23	Missing try/catch On decimals() In Gas Adaptor		
Assets	actions/utils/gas/CommonGasPriceAdaptor.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`CommonGasPriceAdaptor._getConversionExponent()` calls `IERC20Metadata(refundToken).decimals()` directly without a try/catch wrapper, breaking the intended graceful degradation pattern.

```
// @audit direct call can revert for non-standard tokens lacking decimals()
uint256 tokenDecimals = IERC20Metadata(refundToken).decimals();
if (tokenDecimals > 18 + oracleDecimals) {
    return (0, false);
}
```

The adaptors are designed to return `(0, 0)` for unsupported configurations via the `validToken` flag. However, if `refundToken` does not implement the optional `decimals()` function, the external call reverts and propagates through the entire gas refund flow, potentially reverting the enclosing bundle execution.

This issue has a low impact as it can cause gas refund calculation to revert rather than gracefully returning zero. This issue has a low likelihood as it requires a misconfigured or non-standard refund token.

Recommendations

Wrap the `decimals()` call in a try/catch block and return `(0, false)` on failure, consistent with the existing validity-flag pattern.

Resolution

The `decimals()` call was replaced with a low-level `staticcall` that returns `(0, false)` on failure, so a non-standard refund token causes the gas refund to be skipped rather than reverting the bundle. The fix is applied in the shared adaptor base and inherited by all three chain-specific adaptors.

`CommonGasPriceAdaptor._tryGetTokenDecimals()` can still revert if the `decimals()` function returns data that will not `abi.decode` into a `uint8`. Although this is less likely than a completely missing `decimals()`, it would still be possible to get a revert from this function.

Resolved in commit [108b6e4](#). Regarding the possibility of a revert from malformed returned data, the development team chose not to make further changes with the following comment:

We know which gas refund token we are using (USDC). It does report back a correct `decimals()` value.

BRV4-24	bytes4 Pool ID Truncation Enables Collision		
Assets	auth/AdminVault.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`AdminVault._poolIdFromAddress()` truncates pool addresses to a `bytes4` key. This leaves only a 4-byte (32-bit) identifier for each pool, making collisions feasible through CREATE2 address grinding.

```
function _poolIdFromAddress(address _addr) internal pure returns (bytes4) {
    // @audit bytes4 is only a 4-byte key; collisions are feasible with ~2^16 attempts
    return bytes4(keccak256(abi.encodePacked(_addr)));
}
```

An actor holding `POOL_PROPOSER_ROLE` who controls a CREATE2 deployer can grind an address whose `bytes4` pool ID collides with a target pool. Once the colliding address is added, the legitimate pool cannot be registered for the same protocol as `require(protocolPools[protocolId][poolId] == address(0))` reverts.

This issue has a low impact as it enables denial-of-service on pool registration rather than fund loss. This issue has a low likelihood as exploitation requires a privileged role holder and computational grinding effort.

Recommendations

Use the full `address` or `bytes32` hash as the mapping key instead of truncating to `bytes4`. Since pool addresses are already unique identifiers, the truncation provides no benefit beyond marginal gas savings.

Resolution

The development team have closed this issue with the following comment:

The collision is real in isolation, but every step of the path runs through controls already relied upon. The attacker must hold `POOL_PROPOSER_ROLE` (multisig-vetted); `addPool` requires the target slot to be empty, so a colliding entry can only pre-emptively squat an ID, never overwrite or redirect an existing registered pool; and registration is a two-step timelocked flow in which the executor verifies the proposal's full pool address against the vetted deployment before adding — a squatted collision is visible and cancelable throughout the delay window. Changing the key type also cascades beyond `AdminVault` storage into every action's calldata encoding and the offchain signing stack.

BRV4-25 Missing Domain Name And Version Validation In <code>initializeConfig</code>			
Assets	auth/EIP712TypedDataSafeModule.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`EIP712TypedDataSafeModule.initializeConfig()` validates all address parameters as non-zero but accepts empty strings for `_domainName` and `_domainVersion` without any length check.

```
require(
  _adminVault != address(0) &&
  _sequenceExecutor != address(0) &&
  // ... other address checks ...
  _authRegistry != address(0),
  "Invalid input"
);
// @audit _domainName and _domainVersion: no length or content validation
```

Empty strings produce `keccak256("")` for both domain fields in the EIP-712 domain separator. If two module instances are initialised with empty strings and enabled on the same Safe, their domain separators are identical, enabling cross-module signature replay at nonce zero.

This issue has a low impact as the replay window is narrow (only at deployment when both modules share nonce zero). This issue has a low likelihood as it requires a configuration error deploying two modules with empty domain strings on the same Safe.

Recommendations

Add non-empty checks for both `_domainName` and `_domainVersion` alongside the existing address zero-checks.

Resolution

`initializeConfig` now rejects empty `_domainName` or `_domainVersion` in the same guard as the address checks, preventing two modules on one Safe from collapsing to a shared empty-string domain separator. Resolved in commit [b27ad53](#).

BRV4-26	GasRefundAction Fee-On-Transfer Discrepancy		
Assets	actions/utils/GasRefundAction.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`GasRefundAction.executeAction()` transfers the refund token to the EIP712 module without measuring the actual received amount via a balance-diff pattern, causing a discrepancy for fee-on-transfer tokens.

```
uint256 amount = balance < p.maxRefundAmount ? balance : p.maxRefundAmount;
if (amount == 0) return;
IERC20(refundToken).safeTransfer(address(EIP712_MODULE), amount);

// @audit logs nominal 'amount', not actual received amount
LOGGER.logActionEvent(
    LogType.GAS_REFUND_RESERVATION,
    abi.encode(refundToken, address(EIP712_MODULE), amount)
);
```

If `refundToken` is a fee-on-transfer token, the module receives less than `amount` but the log records the full nominal value. Over time, cumulative logged deposits diverge from actual module balance.

This issue has a low impact as the refund token is admin-controlled and expected to be USDC, which is not fee-on-transfer. This issue has a low likelihood as it requires misconfiguration to a fee-on-transfer refund token.

Recommendations

Use a balance-diff pattern: read the module's balance before and after the transfer, and log the actual received amount. Alternatively, document that the refund token must not be fee-on-transfer and enforce via an allowlist.

Resolution

`GasRefundAction` now records the module's balance delta across the refund transfer and logs that instead of the nominal amount, so a fee-on-transfer refund token cannot make the on-chain reservation telemetry diverge from the module's real balance. Resolved in commit [15d1ec3](#).

BRV4-27	ParaswapSwap Source Token Not Validated Against <code>params.tokenIn</code>		
Assets	actions/swap/ParaswapSwap.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

`ParaswapSwap` extracts and validates the destination token from `swapCallData` but performs no equivalent extraction or validation of the source token. The contract approves `params.tokenIn` to the router, but the router acts on whatever token is encoded in the raw calldata, which may differ.

```
address extractedDestToken = extractSwapDestination(params.swapCallData);
// @audit no extractSwapSource – source token in calldata is unchecked

// Approve spending of input token
tokenIn.safeIncreaseAllowance(address(AUGUSTUS_ROUTER), _params.fromAmount);

// Record balance before swap
uint256 balanceBefore = tokenOut.balanceOf(address(this));

// Execute swap through Paraswap
// solhint-disable-next-line avoid-low-level-calls
// @audit router receives raw swapCallData which may reference a different source token
(bool success, ) = address(AUGUSTUS_ROUTER).call(_params.swapCallData);
```

Combined with the allowance accumulation issue in `ParaswapSwap` (which never resets allowances), residual approvals from prior swaps can be consumed by crafted calldata specifying a different source token than declared in `params.tokenIn`.

This issue has a low impact as it enables spending of unintended tokens but only those with pre-existing router allowance, and `tokenOut/minToAmount` checks still apply. This issue has a low likelihood as it requires a malicious authorised operator and relies on accumulated residual allowances.

Recommendations

Extract and validate both the source token and source amount from `swapCallData`, ensuring they match `params.tokenIn` and `params.fromAmount` respectively.

Resolution

`ParaswapSwap` now decodes both the source and destination tokens from the Augustus calldata and requires the source to equal the declared `params.tokenIn`, so crafted calldata cannot route a different input token than the caller declared. Resolved in commit [2aac471](#).

BRV4-28 Co-Signers Can Be Configured With Zero Threshold	
Assets	auth/AuthRegistry.sol auth/EIP712TypedDataSafeModule.sol
Status	Closed: See Resolution
Rating	Informational

Description

The auth registry allows co-signers to be configured while the manager co-sign threshold is zero. In that state the configured co-signers have no effect, because manager-signed bundles can execute without any co-signer approvals. Such a configuration is likely to be accidental and so should be rejected when submitted.

`AuthRegistry._validateSnapshot()` only rejects thresholds above the number of configured co-signers. It does not reject a zero threshold when one or more co-signers are present.

```
function _validateSnapshot(
    uint256 _newVersion,
    address[] memory _newManagers,
    address[] memory _newCoSigners,
    uint256 _managerCoSignThreshold
) private pure {
    // ... snip ...
    // @audit Allows co-signers to be configured with no required co-signatures.
    if (_managerCoSignThreshold > _newCoSigners.length) {
        revert Errors.AuthRegistry_InvalidThreshold(_managerCoSignThreshold, _newCoSigners.length);
    }

    _assertValidMembers(_newManagers, _newCoSigners);
}
```

The module enforces whatever threshold is stored. When the threshold is zero, the co-signature check is satisfied even if no co-signers have approved the bundle.

```
function _verifyThresholds(
    address _safeAddr,
    bool _hasOwner,
    bool _hasManager,
    uint256 _coSignCount
) internal view {
    // ... snip ...
    if (_hasManager) {
        uint256 required = authRegistry.getManagerCoSignThreshold(_safeAddr);
        // @audit A zero threshold makes configured co-signers dormant.
        if (_coSignCount < required) {
            revert Errors.EIP712TypedDataSafeModule_InsufficientCoSignatures(_coSignCount, required);
        }
    }
}
```

This issue is rated as informational because it does not bypass the stored policy: the module correctly enforces the configured threshold. The concern is that a snapshot containing co-signers with a zero threshold is operationally pointless and likely indicates a configuration mistake.

Recommendations

Reject auth snapshots where `_newCoSigners.length > 0` and `_managerCoSignThreshold == 0`. If manager execution without co-signatures is desired, the co-signer list should be empty so the configuration reflects the enforced policy.

Resolution

The development team have closed this issue with the following comment:

A zero threshold with co-signers present is a deliberate “dormant co-signer configuration” used for staged on-boarding: a new co-signer is added so their signatures can be exercised end-to-end without becoming mandatory for live execution; the threshold is raised in a later auth update once the co-signer is verified. Only an unsatisfiable threshold (greater than the co-signer count) is rejected. This is now documented in the registry NatSpec.

The documentation update was made in commit [69f593c](#).

BRV4-29 OP Stack L1 Fee Is Calculated From Incorrect Input Bytes	
Assets	actions/utils/gas/OpStackGasPriceAdaptor.sol auth/EIP712TypedDataSafeModule.sol
Status	Closed: See Resolution
Rating	Informational

Description

The OP Stack adaptor includes the L1 data posting fee as a fixed refund component, which is the correct general approach. However, the module passes `msg.data` to the adaptor as the input for that fee calculation. In this context, `msg.data` is only the ABI calldata for the current `executeBundle()` call.

```
function _getRefundRate() internal view returns (uint256 ratePerGas, uint256 fixedFee) {
    return IGasPriceAdaptor(gasPriceAdaptor).getRefundRate(usdcToken, msg.data);
}
```

OP Stack's `GasPriceOracle.getL1Fee(bytes)` prices the supplied bytes as an unsigned, fully RLP-encoded transaction. Contract calldata is only one field inside that transaction encoding; it omits the transaction envelope and, in relay flows, may also omit calldata from the actual top-level transaction that entered the system.

As a result, the adaptor calculates the fixed L1 data fee from different bytes than the sequencer charges for. The returned `fixedFee` can therefore be inaccurate, and will generally under-refund when the real transaction contains additional envelope or wrapper-call data.

```
(bool ok, bytes memory data) = address(gasPriceOracle).staticcall(
    abi.encodeWithSignature("getL1Fee(bytes)", outerTxCalldata)
);
```

Here, the name `outerTxCalldata` is misleading because the value is not the outer transaction encoding expected by the OP Stack oracle. It is only the current module call's ABI calldata.

This issue is rated as informational because it affects refund accuracy rather than execution correctness. The impact depends on the difference between the calldata passed to `executeBundle()` and the actual transaction bytes paid for by the executor.

Recommendations

Do not pass raw `msg.data` to the OP Stack gas oracle as if it were the full transaction. Prefer passing the unsigned serialized transaction bytes from the relay or offchain execution layer, then call `getL1Fee(bytes)` with that value.

If the full transaction bytes are not available onchain, document the calculation as an approximation and consider using a conservative upper-bound method where supported, such as OP Stack's `getL1FeeUpperBound(uint256)` for the unsigned transaction size.

Resolution

The development team have closed this issue with the following comment:

Passing `executeBundle` calldata rather than the full serialized unsigned transaction makes the L1 component a deliberate under-estimate. The call is `staticcall`-guarded (returns zero on failure) so it can never revert a bundle; the only effect is the protocol slightly under-refunding the executor on OP Stack chains, and the refund is independently capped by the signed `maxRefundAmount`. This is documented as a bounded approximation.

BRV4-30	AcrossV3 Fee Timestamp Key Mismatch	
Assets	actions/ActionBase.sol actions/across-v3/AcrossV3Supply.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

AcrossV3 fee processing records the last fee timestamp under the underlying `l1Token`, while the actual fee-bearing asset is the Across LP token.

```
feeInTokens = _processFee(l1Token, _inputData.feeBasis, lpToken);
```

Inside `ActionBase._processFee()`, the timestamp is read and updated using `_pool`, which is `l1Token` in this case.

```
uint256 lastFeeTimestamp = ADMIN_VAULT.getLastFeeTimestamp(_pool);

if (lastFeeTimestamp == 0) {
    ADMIN_VAULT.setFeeTimestamp(_pool);
    return 0;
}

// ... fee calculation ...

ADMIN_VAULT.setFeeTimestamp(_pool);
```

However, `SendToken` checks fees using the token being transferred.

```
_checkFeesTaken(_tokenAddr);
IERC20(_tokenAddr).safeTransfer(_to, _amount);
```

If `_tokenAddr` is the Across LP token, `_checkFeesTaken(lpToken)` reads a timestamp that was never set. Since the timestamp is 0, the guard passes and the LP tokens can be transferred from the Safe to a Safe owner without paying accrued fees.

```
uint256 feeTimestamp = ADMIN_VAULT.getLastFeeTimestamp(_token);
require(
    feeTimestamp == 0 || feeTimestamp >= block.timestamp,
    Errors.Action_FeesNotPaid(protocolName(), actionType(), _token)
);
```

Recommendations

Use the LP token as the fee timestamp key for AcrossV3 actions, so the key used during fee processing matches the key checked before token transfers.

Resolution

Both `AcrossV3Supply` and `AcrossV3Withdraw` now key the fee timestamp on the LP token (the held position) rather than `l1Token`, so the set side matches the token `_checkFeesTaken` reads on a later transfer. Resolved in commit [5286320](#).

BRV4-31	setActionConfig Allows Immediate Redirection Of Protocol Endpoints	
Assets	auth/AdminVault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

`AdminVault.setActionConfig()` allows an address with `ROLE_MANAGER_ROLE` to immediately update protocol endpoint addresses stored in the `actionConfig` mapping.

```
function setActionConfig(
    string calldata protocolName,
    uint8 actionType,
    address configAddress
) external onlyRole(ROLE_MANAGER_ROLE) {
    actionConfig[protocolName][actionType] = configAddress;
}
```

These config addresses are used by actions through `_configAddress()` to determine critical external protocol contracts, such as lending pools, swap targets, or protocol singletons.

```
function _configAddress() internal view returns (address configAddress) {
    configAddress = ADMIN_VAULT.getActionConfig(protocolName(), actionType());
    require(configAddress != address(0), Errors.InvalidInput(protocolName(), "config"));
}
```

Unlike other sensitive admin operations such as adding pools, adding actions, or updating fee configuration, `setActionConfig()` does not use a proposal, delay, or execution step. As a result, a compromised `ROLE_MANAGER_ROLE` key can immediately redirect an action to a malicious contract. This increases the risk from front running and reentrancy based attacks, amongst others.

Recommendations

Protect `setActionConfig()` with the same two-step timelock pattern used by other privileged configuration changes.

Resolution

The development team have closed this issue with the following comment:

The write is gated by `ROLE_MANAGER_ROLE`, which sits directly under `OWNER_ROLE` in the role hierarchy and carries the strictest key-custody requirements. Every `setActionConfig` emits a monitored `AdminVaultEvent`, config changes are rare and runbook-driven, and the endpoints it configures are consumed by actions that independently validate tokens against the registry. Retro-fitting a timelock requires replacing the deployed `AdminVault`, which is disproportionate for an Informational finding on this trust surface.

BRV4-32	Fee Range Changes Can Affect Already Signed Transactions	
Assets	auth/AdminVault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

User-supplied `feeBasis` values are embedded in signed execution `calldata`, but they are validated against the live `feeConfig` at execution time.

```
function checkFeeBasis(uint256 _feeBasis) external view {
    require(
        _feeBasis >= feeConfig.minBasis && _feeBasis <= feeConfig.maxBasis,
        Errors.AdminVault_FeePercentageOutOfRange(_feeBasis, feeConfig.minBasis, feeConfig.maxBasis)
    );
}
```

If `feeConfig` is changed after a user signs a transaction but before execution, the signed transaction may behave differently than expected.

```
function setFeeConfig() external onlyRole(FEE_EXECUTOR_ROLE) {
    require(pendingFeeConfig.proposalTime != 0, Errors.AdminVault_NotProposed());
    require(
        block.timestamp >= pendingFeeConfig.proposalTime,
        Errors.AdminVault_DelayNotPassed(block.timestamp, pendingFeeConfig.proposalTime)
    );

    LOGGER.logAdminVaultEvent(
        203,
        abi.encode(pendingFeeConfig.recipient, pendingFeeConfig.minBasis, pendingFeeConfig.maxBasis)
    );

    // Update active config; proposalTime is 0 for active config
    pendingFeeConfig.proposalTime = 0;
    feeConfig = pendingFeeConfig;
    delete pendingFeeConfig;
}
```

If `minBasis` is increased, previously valid signed transactions with a lower `feeBasis` can revert. If `maxBasis` is increased, signed transactions using a higher `feeBasis` may become valid under the new range, allowing more fees to be taken than users expected when authorizing their operation.

Recommendations

Bind the expected fee configuration to the user's signed authorization, or otherwise prevent fee range changes from affecting already-signed transactions.

Resolution

The development team have closed this issue with the following comment:

On-chain fees are disabled (`feeBasis == 0`); fee collection is contractual and offchain. The ~10-minute bundle expiry bounds any window in which a fee-range change could affect an in-flight signed bundle.

BRV4-33	ArbitrumGasPriceAdaptor Omits Fixed L1 Fee Component	
Assets	actions/utils/gas/ArbitrumGasPriceAdaptor.sol auth/EIP712TypedDataSafeModule.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

`ArbitrumGasPriceAdaptor.getRefundRate()` stores `arbGasInfo` but never uses it. The function only returns a per-gas refund rate based on `block.basefee` and always sets `fixedFee` to zero.

```
function getRefundRate(
    address refundToken,
    bytes calldata /* outerTxCalldata */
) external view returns (uint256 ratePerGas, uint256 fixedFee) {
    ...
    ratePerGas = _convertPerGasRate(block.basefee, ethUsdPrice, conversionExponent);
    fixedFee = 0;
}
```

The `_executeGasRefund()` in `EIP712TypedDataSafeModule` calculates gas usage with `gasleft()` and then adds `fixedFee`.

```
refundAmount = (gasUsed * ratePerGas) / 1e18 + fixedFee;
```

Because `gasleft()` only measures execution gas, Arbitrum's L1 fee component is not included in `gasUsed`. With `fixedFee = 0`, relayers are under-refunded for Arbitrum bundle executions, and the stored `arbGasInfo` dependency is effectively unused.

Recommendations

Use `arbGasInfo` to include Arbitrum's current transaction L1 fee as the fixed refund component.

Resolution

The development team have closed this issue with the following comment:

On Arbitrum the L1 cost is amortised into `block.basefee`, which the adaptor does use; the uncaptured part is the per-calldata-byte posting component. The effect is a bounded executor under-refund, capped by the signed `maxRefundAmount`, with no user-facing or bundle-reverting impact. This is documented as a bounded approximation.

BRV4-34	getActionConfig Error Omits protocolName	
Assets	auth/AdminVault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

AdminVault.getActionConfig() emits a AdminVault_ConfigNotFound error with a configId derived from only the actionType, discarding the protocolName. Two different protocols sharing the same actionType produce identical error values, making debugging ambiguous.

```
function getActionConfig(string calldata _protocolName, uint8 _actionType) external view returns (address) {
    bytes32 key = keccak256(abi.encodePacked(_protocolName, _actionType));
    address cfg = actionConfig[key];
    // @audit error only hashes _actionType, not _protocolName
    require(cfg != address(0), Errors.AdminVault_ConfigNotFound(bytes4(keccak256(abi.encodePacked(_actionType))));
    return cfg;
}
```

This is rated as informational as the function is read-only and no on-chain state is affected.

Recommendations

Consider including _protocolName in the error's configId so it uniquely identifies the missing entry.

Resolution

The development team have closed this issue with the following comment:

Purely a diagnostic-quality observation: the failing call site always knows the protocolName it passed, so tooling resolves the ambiguity from the transaction context rather than the error payload. No security or funds impact.

BRV4-35	cancelFeeConfigProposal Missing Existence Check	
Assets	auth/AdminVault.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

`AdminVault.cancelFeeConfigProposal()` does not verify that a pending proposal exists before proceeding. When called with no pending fee config, the function emits a misleading cancellation event (event code 303) with all-zero fields and performs a no-op `delete` on an already-zero struct.

```
function cancelFeeConfigProposal() external onlyRole(FEE_CANCELER_ROLE) {
    // @audit no check: pendingFeeConfig.proposalTime != 0
    LOGGER.logAdminVaultEvent(
        303,
        abi.encode(pendingFeeConfig.recipient, pendingFeeConfig.minBasis, pendingFeeConfig.maxBasis)
    );
    delete pendingFeeConfig;
}
```

The sibling `cancelPoolProposal()` correctly guards with `require(poolProposals[proposalId] != 0)`.

This is rated as informational as no state corruption occurs; only the off-chain event trail is polluted.

Recommendations

Consider adding a `proposalTime != 0` guard at the top of the function, consistent with `cancelPoolProposal()`.

Resolution

The development team have closed this issue with the following comment:

Cancelling a non-existent proposal is a harmless no-op on state — it deletes an already-empty struct — and the only artefact is a spurious cancel event with zeroed fields, which the indexer tolerates (it treats cancel events as state resyncs, not deltas). Additionally, onchain fees are disabled (`feeBasis == 0`), so the fee-config path this touches is dormant.

BRV4-36	GasRefundAction Constructor Missing Zero-Address Check	
Assets	actions/utils/GasRefundAction.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

GasRefundAction's constructor does not validate that `_eip712Module` is non-zero before assigning it to the immutable `EIP712_MODULE`. This is inconsistent with sibling contracts `AssignToken` and `UpgradeAction` which both guard their constructor arguments.

```
constructor(
    address _adminVault,
    address _logger,
    address _eip712Module
) ActionBase(_adminVault, _logger) {
    // @audit missing: require(_eip712Module != address(0), ...)
    EIP712_MODULE = IEip712TypedDataSafeModule(_eip712Module);
}
```

A zero `EIP712_MODULE` would cause `safeTransfer` to `address(0)` on every execution, permanently burning refund tokens.

This is rated as informational as it is a deployment-time configuration check and the issue is not exploitable post-deployment.

Recommendations

Consider adding `require(_eip712Module != address(0))` in the constructor, consistent with sibling contracts.

Resolution

The constructor now rejects a zero `_eip712Module` address, matching the zero-address guards on sibling actions. Resolved in commit [07f8047](#).

BRV4-37	PullToken Lacks Delegatecall Context Guard	
Assets	actions/utls/PullToken.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

`PullToken.executeAction()` has no modifier or check ensuring it is only invoked via `delegatecall`. When called directly on the deployed contract, `address(this)` resolves to the `PullToken` contract itself, causing `safeTransferFrom` to pull tokens into an irrecoverable address.

```
// @audit when called directly (not via delegatecall), tokens go to PullToken contract  
IERC20(_tokenAddr).safeTransferFrom(_from, address(this), _amount);
```

Exploitation requires that someone has approved the `PullToken` contract address for a token, which is unlikely in normal operation.

This is rated as informational as realistic on-chain approvals to the `PullToken` contract are uncommon, and the attacker does not profit directly.

Recommendations

Consider adding an `onlyDelegateCall` modifier that verifies `address(this) != SELF` using an immutable self-reference stored at construction time.

Resolution

`PullToken` now carries an `onlyDelegateCall` guard via an immutable self-reference and reverts when called directly on its own deployment. Resolved in commit [c61bd72](#).

BRV4-38	ParaswapSwap Calldata Length Validation Insufficient	
Assets	actions/swap/ParaswapSwap.sol	
Status	Resolved: See Resolution	
Rating	Informational	

Description

`ParaswapSwap.extractSwapDestination()` only checks `_swapCallData.length >= 4`, then reads 32 bytes at offsets up to 164 depending on the selector. For truncated calldata, the assembly `mload` reads beyond the array bounds, producing an incorrect address from adjacent memory.

```
require(_swapCallData.length >= 4, Errors.Paraswap__InvalidCalldata());
// ... destTokenPosition set up to 164 based on selector ...
assembly {
    let dataPtr := add(_swapCallData, 32)
    // @audit reads beyond array bounds if calldata is shorter than destTokenPosition + 32
    destToken := and(mload(add(dataPtr, destTokenPosition)), 0xffffffffffffffffffffffffffffffff)
}
```

In practice, the extracted token is still checked against `TOKEN_REGISTRY` and `params.tokenOut`, so the transaction would revert with a misleading error rather than causing fund loss.

This is rated as informational as malformed calldata produces a misleading error message but no security impact.

Recommendations

Consider adding a minimum length check against the required offset after determining `destTokenPosition`, before the assembly read.

Resolution

The calldata must now be long enough to hold the token words before the assembly read, replacing the bare `length >= 4` check and removing the out-of-bounds read on truncated calldata. Resolved in commit [2aac471](#).

BRV4-39	AssignToken Unvalidated balanceBefore
Assets	actions/utils/AssignToken.sol
Status	Resolved: See Resolution
Rating	Informational

Description

`AssignToken.executeAction()` emits the caller-supplied `params.balanceBefore` into the `BALANCE_UPDATE` event without validating it against any on-chain state. Only `params.balanceAfter` is checked against the actual token balance.

```
LOGGER.logActionEvent(  
  LogType.BALANCE_UPDATE,  
  _encodeBalanceUpdate(  
    _strategyId,  
    _poolIdFromAddress(params.token),  
    params.balanceBefore, // @audit unvalidated caller-supplied value  
    params.balanceAfter,  
    // no fee  
  )  
);
```

An authorised signer can emit arbitrary `balanceBefore` values, causing the off-chain indexer to record false position deltas.

This is rated as informational as no on-chain funds are at risk; only off-chain accounting integrity is affected, and it requires an authorised signer.

Recommendations

Consider documenting that `balanceBefore` is trusted off-chain data, or store the last logged balance on-chain for verification.

Resolution

The development team have resolved this issue with the following comment:

`balanceBefore` is the indexer's prior per-strategy cumulative balance, which has no onchain source of truth to validate against (the chain only knows the Safe's aggregate token holding, not its per-strategy split). It is emitted for offchain delta accounting only; `balanceAfter` remains the sole field validated against actual holdings. This is now documented explicitly in the contract NatSpec.

The documentation update was made in commit [c5051f6](#).

BRV4-40 Miscellaneous General Comments	
Assets	/*
Status	Closed: See Resolution
Rating	Informational

Description

This section details miscellaneous findings discovered by the testing team that do not have direct security implications:

1. Emergency Withdrawal Events May Emit Incorrect Amount

Related Asset(s): *EmergencyWithdrawModule.sol*

The emergency withdraw module emits an event recording the transfer amount of a removed ERC20 token that will be wrong for fee-on-transfer ERC20s.

```
function withdrawERC20(address safe, address token, address to, uint256 amount) external {
    _validateOwners(safe, to);
    uint256 transferAmount = amount;
    if (transferAmount == type(uint256).max) {
        transferAmount = IERC20(token).balanceOf(safe);
    }

    bytes memory data = abi.encodeCall(IERC20.transfer, (to, transferAmount));
    (bool success, bytes memory returnData) = ISafe(safe).execTransactionFromModuleReturnData(token, 0, data,
        ← Enum.Operation.Call);
    if (!success) {
        revert EmergencyWithdraw_TransferFailed();
    }
    if (returnData.length > 0 && !abi.decode(returnData, (bool))) {
        revert EmergencyWithdraw_TransferFailed();
    }

    emit EmergencyERC20Withdrawal(safe, token, to, transferAmount); //@audit the transfer amount here can be wrong
}
```

ERC20s such as PAXG, Paxos' gold backed token, have the Fee-on-Transfer capability which would lead to an incorrect `transferAmount` logged in the `EmergencyERC20Withdrawal` event. This could cause issues for third party integrating bots or client metrics.

As this issue only affects the event emission and poses no security risks it has been logged as informational only.

Add documentation to ensure end users are aware of this behaviour, if Fee-on-Transfer are likely to be used, consider adding different balance validation such as the difference of `token.balanceOf(safe)` before and after the transfer has been executed.

2. Low Level Code Lacks Documentation

Related Asset(s): *AuthRegistry.sol*

Some areas of optimised code would benefit from better documentation about their purpose and mechanism. Such as the bit counting algorithm seen below:

```
function _bitmapsToRestrictions
    address[] memory _managers,
    uint256[] memory _bitmaps
) private pure returns (ITypeD.ManagerRestriction[] memory) {
    // snipped...
    uint256 bitCount;
    uint256 temp = _bitmaps[i];
    while (temp != 0) {
        ++bitCount;
        temp &= temp - 1;
    }

    uint8[] memory actionTypes = new uint8[](bitCount);
    uint256 writeIdx;
    for (uint16 bit; bit < 256; ++bit) {
        if ((_bitmaps[i] >> bit) & 1 == 1) {
            actionTypes[writeIdx++] = uint8(bit);
            if (writeIdx == bitCount) break;
        }
    }

    // snipped...
}

```

Or the allowed action type function:

```
function isActionTypeAllowed(address _safe, address _manager, uint8 _actionType) external view returns (bool) {
    uint256 bitmap = _actionTypeBitmaps[_safe][_manager];
    if (bitmap == 0) return true;
    return ((bitmap >> _actionType) & 1) == 1;
}

```

While the current development team may have a good understanding of these areas better documentation would aid future development by different team members and understanding of third parties making use of Brava's design.

This issue was raised as informational only as no security issues were found in these areas of code.

Improve documentation of complex areas of code to aid future development.

3. Refund Recipient Comment Allows Invalid Values

Related Asset(s): `/auth/EIP712TypedDataSafeModule.sol` `/interfaces/IEip712TypedDataSafeModule.sol`

The NatSpec for `_validateSequenceActionsAndDetectRefund()` describes `_refundRecipient` as non-zero when the bundle expects a gas refund, but the implementation accepts only `0` or `1` when a fee action is present.

```
/// @notice Validates each action in a sequence against its on-chain definition and detects gas refund actions.
/// @param _sequence The sequence whose actions are being validated
/// @param _refundRecipient Non-zero if the bundle expects a gas refund (used to detect refund actions)
/// @return actionIds The validated action ID selectors
/// @return hasRefundAction True if one of the actions is a gas refund action
function _validateSequenceActionsAndDetectRefund(ITypeD.Sequence memory _sequence, uint8 _refundRecipient)
    internal
    view
    returns (bytes4[] memory actionIds, bool hasRefundAction)
{
    // ... snip ...
    if (!hasRefundAction && actualActionType == uint8(IActionBase.ActionType.FEE_ACTION)) {
        // @audit Only 0 or 1 is valid; arbitrary non-zero values revert.
        if !(_refundRecipient == 0 || _refundRecipient == 1) {
            revert Errors.EIP712TypedDataSafeModule_InvalidRefundRecipient(_refundRecipient);
        }
        hasRefundAction = true;
    }
}

```

Update the NatSpec for `_validateSequenceActionsAndDetectRefund()` and `_executeGasRefund()` to specify that `0` refunds `tx.origin`, `1` refunds `FEE_RECIPIENT`, and all other values are invalid when a fee action is present.

4. hasManager Comment Overstates Manager Cardinality

Related Asset(s): /auth/EIP712TypedDataSafeModule.sol

The NatSpec for `_classifySigners()` says `hasManager` is true if at least one signer is a registered manager, but the implementation reverts when a second manager is present. The successful return value therefore means exactly one manager was found, not one or more managers.

```

/// @return hasManager True if at least one signer is a registered manager
/// @return managerAddr The manager address (address(0) if no manager)
/// @return coSignCount Number of co-signer signatures (does NOT include owner or manager)
function _classifySigners(
    address _safeAddr,
    address[] memory _signers
) internal view returns (
    bool hasOwner,
    bool hasManager,
    address managerAddr,
    uint256 coSignCount
) {
    // ... snip ...
    } else if (authRegistry.isManager(_safeAddr, signer)) {
        if (hasManager) {
            // @audit A successful return cannot contain more than one manager.
            revert Errors.EIP712TypedDataSafeModule_MultipleManagers();
        }
        hasManager = true;
        managerAddr = signer;
    }
}

```

Update the NatSpec to state that `hasManager` is true only when exactly one registered manager signer is present, and that the function reverts if multiple manager signatures are supplied.

5. Missing Existence Checks On Action And Pool Cancellation And Removal**Related Asset(s): contracts/auth/AdminVault.sol**

`cancelActionProposal()` and `removeAction()` do not verify that the target entry exists before operating and emitting events. Calling either function with a non-existent or already-removed entry succeeds as a no-op but emits misleading cancellation or removal events to the Logger, polluting the off-chain audit trail.

In contrast, `cancelPoolProposal()` correctly guards with `require(poolProposals[proposalId] != 0)`.

Add existence checks: `require(actionProposals[proposalId] != 0)` in `cancelActionProposal()`, and `require(actionAddresses[_actionId] != address(0))` in `removeAction()`.

6. Zero-Amount Deposit Possible When Vault `maxDeposit` Is Zero**Related Asset(s): contracts/actions/common/ERC4626Supply.sol**

`ERC4626Supply._supplyToVault()` checks `amountToDeposit != 0` before applying the `maxDeposit` cap. If the vault returns `maxDeposit(address(this)) == 0` (paused or at capacity), the amount is capped to zero after the guard, allowing a silent no-op deposit that emits a `BALANCE_UPDATE` event with identical `sharesBefore` and `sharesAfter`.

Add a second zero-check after applying the `maxDeposit` cap, matching the pattern already used in the withdraw path.

Recommendations

Ensure that the comments are understood and acknowledged, and consider implementing the suggestions above.

Resolution

The development team's responses to the raised issues above are as follows:

1. **Emergency Withdrawal Events May Emit Incorrect Amount** — The development team have closed this issue with the following comment:

Telemetry only (the event reports the requested amount, not the post-fee received amount); no funds at risk. We can switch to balance-delta measurement if fee-on-transfer assets become relevant to the emergency path.

2. **Low Level Code Lacks Documentation** — The development team have closed this issue with the following comment:

The most-cited low-level code (the old CCTP snapshot bitmap reconstruction) was removed in the AuthRegistry rewrite (69f593c), so the remaining surface is small; auth-system documentation has been rewritten around the current design (17100ce).

3. **Refund Recipient Comment Allows Invalid Values** — Resolved. The NatSpec was updated to document that 0 and 1 are the only valid non-reverting values for the refund recipient field. Resolved in commit 686c6ff.

4. **hasManager Comment Overstates Manager Cardinality** — Resolved. The NatSpec was corrected to reflect that exactly one manager signature is permitted; a second causes a revert. Resolved in commit 686c6ff.

5. **Missing Existence Checks On Action And Pool Cancellation And Removal** — The development team have closed this issue with the following comment:

All three functions are idempotent no-ops when the target does not exist (deleting empty storage); the only artefact is event noise, which our indexer handles by treating cancel/remove events as state resyncs.

6. **Zero-Amount Deposit Possible When Vault maxDeposit Is Zero** — Resolved. An additional zero-amount check was added after the maxDeposit cap is applied, ensuring a silent no-op deposit can no longer occur when the vault is paused or at capacity. Resolved in commit f9e6604.

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

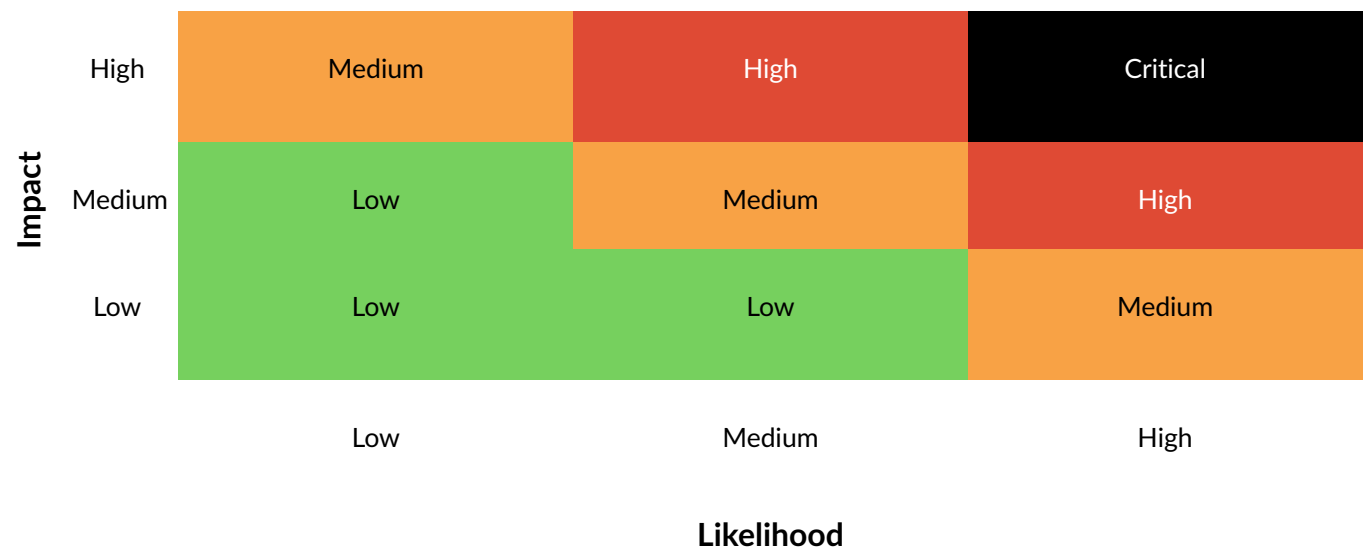


Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'